Wowscanner

Security Scanner

Complete Reference Manual — Version 2.3.0

Professional Edition · Fully Code-Traced Documentation

54 Sections | AES-256 Auth | 6 Output Formats | 4-Layer Integrity

WHAT THIS MANUAL COVERS

21 Reference Sections · 137 Functions Documented · 54 Sections with Exact Weights

Real PASS / FAIL / WARN Messages Traced Directly from Source Code

Complete Cryptographic Specification: AES-256-CTR + PBKDF2-HMAC-SHA256

4-Layer Integrity System · Machine-Bound HMAC · Per-ODF-Member CRC Sidecars

Progress Bar Implementation · EWMA ETA · Weight Recomputation

8 Live Charts · Auth Flow Diagram · Recovery Key Chain · LAN Map

Platform: Debian / Ubuntu · Author: cocoflan · Generated: 25 April 2026

Table of Contents

§	Title	Page
1.	Introduction — Architecture, Design Goals & Version History	3
2.	Script Internals — Globals, Colours, Output Functions & Helpers	5
3.	Requirements, Runtimes & Auto-Installed Tools (16 tools)	7
4.	First-Run Setup — _ws_first_run_setup() Step by Step	8
5.	All Commands — Full Code Reference	9
6.	Flags, Environment Variables & Weight Recomputation	13
7.	All 54 Sections — Label, Function, Weight, Exact Checks	14
7.1	Pentest 0a–0e — Tools, Invocations, PASS/FAIL/SKIP	14
7.2	Core Sections 1–16 — Real Thresholds & PASS/FAIL Messages	15
7.3	Extended Sections 17b–17z — Every Sub-Check Described	20
8.	Kernel Hardening — cmd_harden() — All 21 sysctl Parameters	26
9.	Risky Ports — KNOWN_RISKY / RISKY_PORTS — All Entries	27
10.	Risky Kernel Modules — _risky_desc — All 15 Entries	27
11.	Authentication System — Complete Cryptographic Specification	28
11.1	_ws_crypto() Python Heredoc — Full Code Walkthrough	28
11.2	auth.key — All 10 Fields, Types & Derivation	29
11.3	_ws_verify_pass() — 5-Step Code Path (L502)	30
11.4	_ws_verify_recovery_key() — Hash Chain (L482)	31
11.5	cmd_reset_auth() — All 7 Sub-Commands	31
11.6	Troubleshooting Authentication — 7 Scenarios	32
12.	Output Files — All 9 Formats, Generators & ODT Contents	33
13.	Score Formula, Rating Thresholds & Regression Detection	34
14.	Persistent Data — All 11 Files with Format & Update Function	35
15.	Progress Bar & EWMA ETA — Implementation Detail	36
16.	Report Delivery — deliver_report() & archive_outputs()	37
17.	Generated File Examples with 8 Live Charts	38
18.	Integrity & CRC System — 4 Layers with Attack Matrix	45
19.	Scheduling & Automation	49
20.	Troubleshooting — 16 Scenarios with Code References	50
21.	Quick Reference Card	51

54 Sect

8 Live Charts · Auth Flow Diagram · Recovery Key Chain · LAN Map

Platform: Debian / Ubuntu · Author: cocoflan · Generated: 25 April 2026

1. Introduction — Architecture, Design Goals & Version History

Wowscanner is a production-grade, single-file Linux security audit tool. The entire tool is 16,495 lines of Bash with 137 defined functions. Six output formats are generated using embedded Python 3 heredocs — no external templating engine, no web framework, no additional packages beyond Python stdlib. The tool is portable: copy the script to any Debian or Ubuntu system and run it as root.

Architecture Layers

Layer	What It Does	Key Code
CLI & Arg Parsing	Loop over $@ before anything else. Sets CMD_* and USE_* boolean flags. Pre-scans $@ for bypass keywords before calling _ws_early_gate.	Lines 244–330. Arg loop + pre-check block that sets _WS_BYPASS_AUTH.
Auth Gate	_ws_early_gate() fires immediately after arg parsing for every invocation. Calls _ws_prompt() which calls _ws_verify_pass() via _ws_crypto Python heredoc.	_ws_early_gate() L700, _ws_prompt() L644, _ws_verify_pass() L502, _ws_verify_recovery_key() L482, _ws_crypto() L330.
Command Dispatcher	If any CMD_* flag is true, runs the corresponding cmd_*() function then exits immediately. Falls through to main() only for scan runs.	Lines 16380–16495. One if/then block per CMD_* flag in priority order.
main() Scan Engine	Calls _recompute_progress_cum(), _progress_init(), prints startup ASCII-art banner, then runs 54 _progress_start/section_fn/elapsed triplets.	main() L15975. _recompute_progress_cum() L15985. 54 dispatches with || true guard.
Section Functions (62)	Each section_*() calls pass()/fail()/warn()/info()/detail()/skip(). All output goes through log() which tees to $REPORT and terminal.	L2139–L15459. 62 section functions across all security domains.
Report Generators (6)	After all sections: generate_odt_report(), generate_stats_ods(), generate_odf_intel_report(), generate_html_report(), generate_findings_report(), archive_outputs().	L7666, L8500, L5682, L11593, L13099, L15460

Design Goals & Implementation

Goal	Implementation Detail
Single file, zero install	16,495-line Bash script. No apt packages needed at runtime beyond python3 (stdlib only). Tool auto-installs its own audit tools via apt-get on first use.

Goal	Implementation Detail
Passphrase gate on every run	_ws_early_gate() fires before any section, any command, any output. Root cannot bypass it — the decryption key derives from the passphrase via PBKDF2. Root is only the ownership proof for the forgot/force/rk reset paths.
Recovery from forgotten passphrase	Three recovery paths: (1) reset-auth forgot — root access proves ownership; (2) reset-auth rk — 48-char recovery key from first-run; (3) wowscanner_recover.sh standalone tool that deletes auth.key directly.
54 structured security sections	Covers: pentest probes, users, SSH, firewall, ports, permissions, services, logging, kernel, cron, packages, hardening, containers, rootkits, MAC, Lynis, LAN discovery, port scan, plus 29 extended sections.
6 output formats auto-generated	ODT with 7 SVG charts, HTML (self-contained), ODS 8-sheet workbook, plain-text audit log, findings report, ZIP archive. All generated after every scan without any extra commands.
Persistent scan history	score_history.db (trend chart), timing_history.db (EWMA ETA), port_history.db (new port detection), findings_last.db (regression detection), port_issues.log (cross-run port tracking).
4-layer file integrity	Layer 1: per-file SHA-256+SHA-512 sidecars. Layer 2: per-ODF-member CRC sidecars. Layer 3: HMAC-SHA256-signed INTEGRITY.txt inside ZIP. Layer 4: archive SHA-256 sidecar. Machine-bound HMAC key.
No telemetry, no network calls	Zero outbound connections during scan. No data leaves the machine. Optional --webhook and --email flags send only after scan completes.

Version History — All Releases

Version	Key Changes
v1.1.0	section_supply_chain (pip/npm audit), section_immutable (chattr +i), section_proc_exposure (/proc leaks), baseline command, --email/--webhook flags
v1.2.0	section_failed_logins, section_env_security, adaptive subheader width, visual score bar, progress bar gradient, paginated findings report, SHA-256+SHA-512 .txt sidecars, tab-completion, rich cmd_help panel
v1.3.0	12 new sections (17k–17v): PAM, filesystem, containers, repos, time-sync, IPv6, SSH extras, core dumps, systemd sandboxing, sudo audit, log integrity, compilers. ODS All Findings sheet. Findings .txt box-detection parser.
v1.4.0	4 new sections (17w–17z): network interfaces, kernel modules, MAC profiles, exposure summary. PYEOF parser fixes. STATSEOF truncation removed.
v1.5.0	ODT complete rewrite: 3-colour section headings, detail lines inline, improved SVG charts. Password policy: empty field + SHA-512 checks. SSH: StrictModes + AllowAgentForwarding.
v1.6.0	section_hardware_security (Secure Boot, IOMMU, CPU vulns, TPM), section_boot_security (GRUB pw, cmdline audit), section_systemd_hardening (expanded sandboxing), section_kernel_modules (15 risky modules), improved promiscuous detection.
v2.0.0	AES-256 passphrase auth system: _ws_early_gate(), _ws_prompt(), _ws_verify_pass(). PBKDF2-HMAC-SHA256. auth.key format v1 with HMAC integrity. Replaced legacy set-password system entirely.

Version	Key Changes
v2.1.0	data_key wrapping system (dk_iv + dk_wrapped in auth.key). key_id fingerprint. Normal passphrase change preserves data_key — all archives remain readable. auth.key format v2.
v2.2.0	Recovery key at first-run (rk_hash in auth.key). cmd_reset_auth() interactive menu (6 options). clean --all wipes /etc/wowscanner/. example-output command. debsums auto-install. clean --full factory reset.
v2.3.0	Critical bug fix: removed dead _auth_prompt/_auth_gate/_auth_hash/_auth_crc code (L1949) that caused SECURITY ALERT on every run. Gate now checks raw $@ before variable lookup. _ws_verify_recovery_key() 48-char length check. Dispatcher fixed.

2. Script Internals — Globals, Colours, Output Functions & Helpers

Key Global Variables (set before arg parsing, L174–310)

Variable	Default / Value	L#	Purpose
VERSION	"2.3.0"	L174	Script version — banners, report headers, auth.key comments
PROGRAM	"Wowscanner Security Scanner"	L175	Display name
TIMESTAMP	date +%Y%m%d_%H%M%S	L792	Stamped onto every output filename at scan start
REPORT	"wowscanner_${TIMESTAMP}.txt"	L793	Primary output — every log() call tee-appends here
LAN_JSON	"/tmp/wowscanner_lan_${TIMESTAMP}.json"	L794	LAN discovery output; fed to ODT LAN map SVG
SCORE	0	L795	Incremented by pass() only. Numerator of score.
TOTAL	0	L796	Incremented by pass(), fail(), warn(). Denominator.
USE_PENTEST	true	L203	Controls sections 0a-0e. Set false by --no-pentest.
USE_LYNIS	true	L202	Controls section 15. Set false by --no-lynis.
USE_RKHUNTER	true	L204	Controls 14b rkhunter run. Set false by --no-rkhunter.
USE_HARDENING	true	L205	Controls section 13c (688 lines). Set false by --no-hardening.
USE_NETCONTAINER	true	L206	Controls section 13d. Set false by --no-netcontainer.
USE_FAST_ONLY	false	L207	When true: sections self-limit slow sub-operations.
QUIET	false	L208	When true: info() becomes a no-op. Pass/fail/warn still logged.
PERSIST_DIR	"/var/lib/wowscanner"	L799	All persistent DB files. Created by init_persist_store() L6874.
_WS_AUTH_DIR	"/etc/wowscanner"	L325	Auth directory. Created mode 700 at first-run.
_WS_AUTH_KEY	"${_WS_AUTH_DIR}/auth.key"	L326	Passphrase key file — mode 400, root:root.

Variable	Default / Value	L#	Purpose
WEBHOOK_URL	"${WEBHOOK_URL:-}"	L239	HTTP POST target after scan. Overrideable via env.
REPORT_EMAIL	"${REPORT_EMAIL:-}"	L240	Email recipient for archive. Overrideable via env.
APT_UPDATED	0	L760	Guards maybe_apt_update() — avoids duplicate apt-get update.
APT_CACHE_MAX_AGE	86400	L769	Max age of apt cache in seconds before forced refresh.

ANSI Colour System (L180–199)

Variable	ANSI Code	Used For
RED	'\033[0;31m'	Error messages, secondary fail colour
BRED	'\033[1;31m'	[✘ FAIL] icon, critical headers, progress bar fill
YELLOW	'\033[1;33m'	[■ WARN] icon
GREEN	'\033[0;32m'	Score bar when GOOD rating
BGREEN	'\033[1;32m'	[✔ PASS] icon, Ω delimiter on pass lines
CYAN	'\033[0;36m'	Section subheaders, general information text
BCYAN	'\033[1;36m'	[■ INFO] icon, main startup banner border
BBLUE	'\033[1;34m'	ASCII art WOW title lines in startup banner
BMAGENTA	'\033[1;35m'	Detail line ■ arrows
ORANGE	'\033[0;33m'	[● NOTE] icon
DIM	'\033[2m'	Detail lines body, disabled mode flags, secondary text
BOLD	'\033[1m'	Section headers, emphasis
NC	'\033[0m'	Reset — used at end of every coloured string

Core Output Functions — Defined L872–878 (Verbatim)

```
pass()   { SCORE=$((SCORE+1)); TOTAL=$((TOTAL+1));
           log "  ${BGREEN}[✔ PASS]${NC}  $1  ${BGREEN}Ω${NC}"; }
fail()   { TOTAL=$((TOTAL+1));
           log "  ${BRED}[✘ FAIL]${NC}  $1  ${BRED}Ω${NC}"; }
warn()   { TOTAL=$((TOTAL+1));
           log "  ${YELLOW}[■ WARN]${NC}  $1  ${YELLOW}Ω${NC}"; }
info()   { log "  ${BCYAN}[■ INFO]${NC}  $1  ${BCYAN}Ω${NC}"; }   # no-op when QUIET=true
detail() { log "  ${DIM}■${NC}       ${BMAGENTA}■${NC} $1"; }      # indented sub-line, no score effect
skip()   { log "  ${DIM}[■ SKIP]${NC}  $1  ${DIM}Ω${NC}"; }       # tool missing / service inactive
note()   { log "  ${ORANGE}${BOLD}[● NOTE]${NC}  $1  ${ORANGE}Ω${NC}"; }  # contextual note
log()    { echo -e "$*" | tee -a "$REPORT" || true; }  # terminal + append to REPORT
```

Info: The Ω character at the end of every PASS/FAIL/WARN/INFO/SKIP line is a machine-parseable record delimiter. generate_findings_report() (L13099) uses the regex `sed -n "s/.*\[✘ FAIL\] //;s/ Ω.*//p"` to extract clean finding text for the findings summary file. show_finding_delta() (L11248) uses the same pattern for regression detection.

header() and subheader() — Adaptive Width

```
# header() — full-width ■■■ box that adapts to terminal width:
header() {
  local _bw=$(( ${_PROGRESS_COLS:-80} - 4 ))
  # ... prints ■■■...■■■ box with section title centred ...
}

# subheader() — ■■ label ■■■■■■■■■■■■■... adaptive fill:
subheader() {
  local _t="$1"
  local _w=$(( ${_PROGRESS_COLS:-80} - 6 ))
  local _fi=$(( _w - ${#_t} - 4 ))
  printf -v _fill "%*s" "$_fi" ""; _fill="${_fill// /■}"
  log "  ${BCYAN}■■ ${BOLD}${_t}${NC}${BCYAN} ■${_fill}${NC}"
}
```

3. Requirements, Runtimes & Auto-Installed Tools

Requirement	Minimum	Recommended / Notes
OS	Debian 10 / Ubuntu 18.04	Ubuntu 22.04 LTS / Debian 12 (both tested)
Shell	bash 4.2+ (associative arrays required)	bash 5.x
Python	3.6+ (stdlib only: hashlib, hmac, binascii, os, zipfile)	3.10+
Privileges	sudo access	Root uid=0 (section 8 find operations require it)
Disk	50 MB for outputs	200 MB (full ODT + ODS + HTML + archive)
RAM	128 MB free	512 MB free (rkhunter scan is memory-intensive)
Network	Not required during scan	Optional: apt-get for tool auto-install

Typical Runtimes (from script header comments, L47–57)

Mode / Flags	Typical Duration	Sections Active
Default (no flags, full pentest)	8-15 min	All 54 including active probes
--no-pentest	3-6 min	49 sections (0a-0e skipped, weights[0-4]=0)
--no-pentest --no-lynis	2-4 min	48 sections
--fast-only	2-4 min	All 54 — slow sub-checks skipped per section
--fast-only --no-pentest --no-lynis --quiet	~2 min	Fastest useful scan
LYNIS_FULL=true RKH_FULL=true	10-20 min	Full Lynis + full rkhunter depth

Auto-Installed Tools — All 16 with Exact Invocations

Tool	Sections	wt	Auto-Install	Exact Invocation in Script
nmap	0a, 16	90/30	apt-get install -y nmap	nmap -sV -O -A --script=banner,http-title,ssh-hostkey ... 127.0.0.1
nikto	0b	120	apt-get install -y nikto	timeout 120 nikto -h "${SCHEME}://127.0.0.1:${PORT}" -C all -output ...
hydra	0c	60	apt-get install -y hydra	timeout 60 hydra -L /tmp/ws_users.txt -P /tmp/ws_pass.txt ssh://127.0.0.1:$PORT

Tool	Sections	wt	Auto-Install	Exact Invocation in Script
sqlmap	0d	90	apt-get install -y sqlmap	timeout 90 sqlmap -u "http://127.0.0.1:${HTTP_PORT}/?id=1" --batch --level=2 --risk=2
stress-ng	0e	25	apt-get install -y stress-ng	timeout 30 stress-ng --cpu 2 --vm 1 --io 1 --timeout 15s --metrics-brief
hping3	0e	25	apt-get install -y hping3	timeout 10 hping3 -S --flood -p 80 127.0.0.1 2>/dev/null &
enum4linux	0a	90	apt-get install -y enum4linux	timeout 30 enum4linux -a 127.0.0.1 2>/dev/null
chkrootkit	14b	70	apt-get install -y chkrootkit	chkrootkit -q 2>/dev/null
rkhunter	14b	70	apt-get install -y rkhunter	rkhunter --check --sk --nocolors --rwo 2>/dev/null
debsums	13	35	apt-get install -y debsums	debsums -c 2>/dev/null (checks md5 of every installed package file)
lynis	15	50	apt-get install -y lynis	lynis audit system --quick --fast --no-colors 2>/dev/null
arp-scan	16a	25	apt-get install -y arp-scan	arp-scan --localnet 2>/dev/null (fallback: nmap -sn; /proc/net/arp)
lsof	17i	4	apt-get install -y lsof	lsof -n -P 2>/dev/null (deleted executables, sensitive open files)
mokutil	17b3	6	apt-get install -y mokutil	mokutil --sb-state 2>/dev/null (Secure Boot state check)
getcap	13c	28	apt-get install -y libcap2-bin	getcap -r / 2>/dev/null (Linux capability audit)
openssl	13c	28	usually pre-installed	openssl x509 -enddate; openssl s_client for TLS protocol checks

4. First-Run Setup — _ws_first_run_setup() L567

First run is detected inside _ws_early_gate() (L700) by the check `[[ ! -f "$_WS_AUTH_KEY" ]]`. If true and root, _ws_first_run_setup() is called. If not root, the gate prints the sudo command and exits 1. The wizard displays a full-width BCYAN ■■■ box at each step.

Step	Code in _ws_first_run_setup()	What Happens
1. Root check	[[$(id -u) -ne 0]] → echo sudo cmd; exit 1	Prints: "Run as root: sudo bash $0". Does not self-elevate.
2. Auth dir creation	mkdir -p "$_WS_AUTH_DIR"; chmod 700; chown root:root	/etc/wowscanner/ created mode 700. Only root can enter or list it.
3. Passphrase entry	read -rsp loop × 2. ${#_p1} -lt 10 → "too short, try again"	Min 10 chars enforced. Passphrase never stored — passed to _ws_write_auth_key(). Wiped: _p1=""; _p2="".
4. Key generation	_ws_write_auth_key "$_p1" "" (empty 2nd arg = generate new data_key)	PBKDF2 200,000 rounds. Random 32-byte data_key generated. Writes auth.key mode 400. Exports WS_RECOVERY_KEY.
5. Recovery key display	BRED ■■■ box. read -rp "Press Enter to confirm you saved it..."	48-char hex shown split as 4×12 chars. Wizard blocks until Enter. Key is sha256(master_key+b"RECOVERY")[:48].
6. Sudoers (optional)	read -rp "Install NOPASSWD sudoers entry? [y/N]"	Creates /etc/sudoers.d/wowscanner with NOPASSWD:SETENV for this script path.

```
# Recovery key display (L567+):
echo -e "${BRED}${BOLD}■■■
```

5. All Commands — Full Code Reference

(no command) — Full Scan → main() L15975

The default when no CMD_* flag is set. main() is the last function called in the script. It calls require_root(), then check_archive_presence() (warns immediately if a known ZIP has disappeared), records T_START and _T_START_SECONDS, calls _recompute_progress_cum(), _progress_init(), prints the adaptive ASCII-art startup banner, then dispatches all 54 sections in order.

```
# Dispatch pattern — identical for all 54 sections:
_progress_start "8 permissions"    # draws bar at floor%, shows label RUNNING
section_permissions || true        # || true: crash never aborts scan
_elapsed "8 permissions"           # records wall-clock, advances bar to ceil%

# Startup banner adapts to terminal width (stored in _PROGRESS_COLS):
log "${BCYAN}■${NC}$(_bpad "■■■    ■■■ ■■■■■■■ ■■■    ■■■■■■■■■■■")${BCYAN}■${NC}"
log "${BCYAN}■${NC}$(_lpad "  ■  Host    :  ${WHITE}${_hostname}${NC}${BCYAN}")${NC}"
log "${BCYAN}■${NC}$(_lpad "  Pentest ${_m_pen_c}${_m_pentest}${NC}  Lynis ${_m_lyn_c}${_m_lynis}...")${NC}"
```

clean → cmd_clean() L12701

Prints a ■■■ header box then uses `find "$PWD" -maxdepth 1 -name "wowscanner_*"` to collect output files. For each: records size via stat, removes, accumulates _bytes total. Prints summary table of what was deleted and total space freed.

Variant	Extra Actions After File Deletion
clean	Only CWD wowscanner_* output files deleted.
clean --all	Also: rm -rf "$PERSIST_DIR" (/var/lib/wowscanner/ — all history), rm -rf "$_WS_AUTH_DIR" (/etc/wowscanner/ — auth.key), rm -f /etc/sudoers.d/wowscanner. Next run triggers first-run wizard.
clean --full	Everything that --all does PLUS CWD output files. Complete factory reset. Equivalent to a fresh install.
clean --integrity	Only rm -f "$PERSIST_DIR/integrity_alerts.log". Leaves all other data intact.

```
sudo bash wowscanner.sh clean
sudo bash wowscanner.sh clean --all      # wipe auth + history
sudo bash wowscanner.sh clean --full     # complete factory reset
```

verify → cmd_verify() L12175

Finds the most recent archive ZIP via `find -printf "%T@\t%p\n" | sort -rn | head -1`. Calls verify_txt_crc() for text sidecars, check_odf_crcs() for ODF sidecars, then opens the ZIP and validates INTEGRITY.txt HMAC using the machine key. Every file is verified in 64 KB streaming chunks. See Section 18 for full detail.

```
sudo bash wowscanner.sh verify

# Example output:
  ■■■■■■■■■■■
```

```
■   Wowscanner Security Scanner  v2.3.0                    ■
■■■
```

Called when CMD_RESET_AUTH=true. Normalises the sub-command by stripping leading dashes and lowercasing. If empty, shows the numbered menu. **Gate bypass**: _ws_early_gate() (L700) checks raw $@ in a for loop before any variable lookup — forgot, force, rk, keyid, recover all bypass the passphrase prompt entirely.

```
# Sub-command normalisation (L1650-1658):
local _sub="${1:-}"
[[ "$_sub" == "reset-auth" ]] && _sub="${2:-}"  # strip "reset-auth" if first arg
_sub="${_sub#--}"; _sub="${_sub#-}"; _sub="${_sub,,}"  # strip dashes, lowercase

# _ws_early_gate() bypass check (L700) — raw $@ scan before any variable:
for _ga in "$@"; do
  case "$_ga" in
    forgot|--forgot|force|--force|rk|--rk|recovery-key) return 0 ;;
  esac
done
if [[ "${1:-}" == "reset-auth" ]]; then
  case "${2:-}" in forgot|force|rk|keyid|recover|"") return 0 ;; esac
fi
```

Sub-command	Root?	Old Pass?	data_key After	Key ID After	Archives
(menu)	YES	—	—	—	Shows options 1-6
change	YES	YES	Preserved	Same	All readable — passphrase changed, data_key untouched
forgot	YES	NO	NEW	NEW	Old archives need recover path. auth.key.prev kept.
force	YES	NO	DELETED	GONE	auth.key.prev kept for recover path.
rk	YES	NO	NEW	NEW	Old archives need recover path. auth.key.prev kept.
keyid	NO	NO	Unchanged	Shown	Read-only. No changes.
recover	NO	OLD	Unchanged	From .prev	Exports WS_DATA_KEY from auth.key.prev

example-output → cmd_example_output() L913

Generates two realistic example files without running any scan and without requiring auth. _ws_early_gate() returns 0 for CMD_EXAMPLE_OUTPUT before any passphrase check. A 700-line Python heredoc writes realistic output for a fictional Ubuntu 22.04 web server.

```
bash wowscanner.sh example-output
# Creates:
#   wowscanner_example_TIMESTAMP.txt          (735 lines, ~46 KB)
#   wowscanner_example_findings_TIMESTAMP.txt (57 lines, ~4.5 KB)
```

install-timer / remove-timer → cmd_install_timer() L11391

```
# Creates two systemd unit files:
# /etc/systemd/system/wowscanner.service
# /etc/systemd/system/wowscanner.timer  (OnCalendar=weekly = Sun 00:00 UTC)
```

```
sudo bash wowscanner.sh install-timer
systemctl daemon-reload
systemctl enable --now wowscanner.timer
systemctl list-timers wowscanner.timer
journalctl -u wowscanner.service --since "24 hours ago"

sudo bash wowscanner.sh remove-timer
```

install-completion → cmd_install_completion() L12036

Writes a bash completion script to `/etc/bash_completion.d/wowscanner`. The _wowscanner_complete() function (L12050) completes command names and flag names on Tab. Takes effect immediately in new shells.

```
sudo bash wowscanner.sh install-completion
# Then source /etc/bash_completion or open a new shell:
sudo bash wowscanner.sh [TAB]    # shows: clean diff harden baseline verify ...
```

6. Flags, Environment Variables & Weight Recomputation

All Flags — Internal Variable, Weight Effect, Time Saved

Flag	Sets Variable	Weight Effect in _recompute_progress_cum()	Time Saved
--no-pentest	USE_PENTEST=false	WEIGHTS[0..4]=0 (removes 385 of 925 pts)	5-12 min
--no-lynis	USE_LYNIS=false	WEIGHTS[22]=0 (removes 50 pts)	30s-5 min
--no-rkhunter	USE_RKHUNTER=false	WEIGHTS[20]=0 (removes 70 pts)	1-3 min
--no-hardening	USE_HARDENING=false	WEIGHTS[18]=0 (removes 28 pts)	~45s
--no-netcontainer	USE_NETCONTAINER=false	WEIGHTS[19]=0 (removes 18 pts)	~20s
--fast-only	USE_FAST_ONLY=true	No weight change — sections self-limit	2-5 min
--quiet	QUIET=true	No weight change — info() becomes no-op	0s
--email=addr	REPORT_EMAIL=addr	No weight change — fires in deliver_report()	0s
--webhook=url	WEBHOOK_URL=url	No weight change — fires in deliver_report()	0s

Environment Variable Overrides (set before sudo -E)

Variable	Default	Effect When Set
LYNIS_FULL	unset (fast mode)	Set "true" → run lynis without --quick --fast (adds 2-10 min)
RKH_FULL	unset (fast mode)	Set "true" → run rkhunter without --skip-keyfile-check (adds 1-5 min)
APT_CACHE_MAX_AGE	86400 (1 day)	Set 0 → force apt-get update before every tool install
WEBHOOK_URL	empty	Equivalent to --webhook= flag. Overridden by flag if both set.
REPORT_EMAIL	empty	Equivalent to --email= flag. Overridden by flag if both set.

```
LYNIS_FULL=true RKH_FULL=true sudo -E bash wowscanner.sh
APT_CACHE_MAX_AGE=0 sudo bash wowscanner.sh --no-pentest
```

_recompute_progress_cum() — Weight Recomputation (L15985)

```
# Called once per skipped module to keep running total accurate:
_recompute_progress_cum() {
  _pw_sum=0
  for (( _pw_i=0; _pw_i<${#_PROGRESS_WEIGHTS[@]}; _pw_i++ )); do
    _pw_sum=$(( _pw_sum + _PROGRESS_WEIGHTS[_pw_i] ))
    _PROGRESS_CUM[$_pw_i]=$_pw_sum   # prefix sum for floor/ceil calc
  done
  _PROGRESS_TOTAL=$_pw_sum           # updates running total
}

# In main(): called once per skipped module:
if [[ "$USE_PENTEST" != "true" ]]; then
  for _pi in 0 1 2 3 4; do _PROGRESS_WEIGHTS[$_pi]=0; done
  _recompute_progress_cum   # total drops from 925 to 540
fi
if [[ "$USE_LYNIS" != "true" ]]; then
  _PROGRESS_WEIGHTS[22]=0; _recompute_progress_cum   # -50 pts
fi
```

7. All 54 Sections — Label, Function, Weight, Exact Checks

The table below maps the progress label (used in _progress_start), the function name, the ETA weight in the 925-point total, and the cumulative weight. Every section is dispatched as: `_progress_start "label"; section_fn || true; _elapsed "label"`. The `|| true` ensures a section crash never aborts the scan. _elapsed records the wall-clock seconds to timing_history.db and advances the bar. The weight controls how much of the progress bar the section fills, not the number of checks it runs.

#	Label	Function (L#)	wt	cum	Skip Flag
0	0a pentest-enum	section_pentest_enum (L2139)	90	90	--no-pentest
1	0b pentest-web	section_pentest_web (L2361)	120	210	--no-pentest
2	0c pentest-ssh	section_pentest_ssh (L2413)	60	270	--no-pentest
3	0d pentest-sqli	section_pentest_sqli (L2481)	90	360	--no-pentest
4	0e pentest-stress	section_pentest_stress (L2527)	25	385	--no-pentest
5	1 sysinfo	section_sysinfo (L2688)	3	388	—
6	2 updates	section_updates (L2798)	18	406	—
7	3 users	section_users (L2860)	12	418	—
8	4 password	section_password_policy (L3014)	3	421	—
9	5 ssh	section_ssh (L3078)	6	427	—
10	6 firewall	section_firewall (L3250)	3	430	—
11	7 ports	section_ports (L3298)	4	434	—
12	8 permissions	section_permissions (L3417)	55	489	—
13	9 services	section_services (L3540)	4	493	—
14	10 logging	section_logging (L3607)	8	501	—
15	11 kernel	section_kernel (L3772)	5	506	—
16	12 cron	section_cron (L3843)	5	511	—
17	13 packages	section_packages (L3989)	35	546	—
18	13c hardening-advanced	section_hardening_advanced (L4213)	28	574	--no-hardening
19	13d network-container	section_network_container (L4906)	18	592	--no-netcontainer
20	14b chkrootkit+rkhunter	section_chkrootkit (L5137)	70	662	--no-rkhunter

#	Label	Function (L#)	wt	cum	Skip Flag
21	14 mac	section_mac (L5370)	3	665	—
22	15 lynis	section_lynis (L5423)	50	715	--no-lynis
23	16a lan-scan	section_lan_scan (L6986)	25	740	—
24	16 portscan	section_portscan (L7358)	30	770	—
25–54	17b–17z (29 extended sections)	section_failed_logins … section_exposure_summary	155	925	—

7.1 Pentest Sections 0a–0e — Exact Tools & Conditions

ID wt	Function	Exact Invocation, Tool, PASS / FAIL / SKIP Conditions
0a 90	section_pentest_enum	nmap: nmap -sV -O -A --script=banner,http-title,ssh-hostkey ... 127.0.0.1. enum4linux: timeout 30 enum4linux -a 127.0.0.1 (SMB null session, shares, users). PASS: "nmap enumeration completed" — clean scan. WARN: "nmap produced no output" — scan may have been blocked. WARN: "nmap could not be installed — skipping enumeration". INFO: every open port with version string, OS guess, banner. Results stored for cross-reference with section 16 portscan and Intel ODT report.
0b 120	section_pentest_web	nikto: timeout 120 nikto -h "${SCHEME}://127.0.0.1:${PORT}" -C all -output /tmp/nikto_$PORT.txt. Run against each detected HTTP/HTTPS port from nmap. FAIL: "Nikto found $NIKTO_VULNS finding(s) on port $PORT" — each listed with detail(). PASS: "Nikto: No critical findings on port $PORT". PASS: "No web server listening (Nikto not needed)" — no HTTP ports detected. WARN: "nikto could not be installed — skipping web scan". Heaviest section by weight (120) — large site scans take longest.
0c 60	section_pentest_ssh	hydra: timeout 60 hydra -L /tmp/ws_users.txt -P /tmp/ws_pass.txt ssh://127.0.0.1:$PORT. 10 credential pairs generated by the section: root/toor, admin/admin, user/password, pi/raspberry, ubuntu/ubuntu, test/test, oracle/oracle, mysql/mysql, ftp/ftp, guest/guest. FAIL (CRITICAL): any login succeeds — credential pair listed with detail(). PASS: "SSH brute-force test not applicable" if no SSH port detected. WARN: "hydra could not be installed — skipping SSH brute-force test".
0d 90	section_pentest_sqli	sqlmap: timeout 90 sqlmap -u "http://127.0.0.1:${HTTP_PORT}/?id=1" --batch --level=2 --risk=2. FAIL: "sqlmap found SQL injection vulnerability on port $HTTP_PORT!" — DB type listed. PASS: "sqlmap: No SQL injection found on port $HTTP_PORT (basic probe)". PASS: "SQL injection test not applicable (no web server found)". WARN: "sqlmap could not be installed — skipping SQLi probe".
0e 25	section_pentest_stress	stress-ng: timeout 30 stress-ng --cpu 2 --vm 1 --io 1 --timeout 15s --metrics-brief. hping3: timeout 10 hping3 -S --flood -p 80 127.0.0.1 2>/dev/null & (SYN flood to loopback). Records /proc/stat OOM kill count before/after. Checks sysctl net.ipv4.tcp_syncookies (SYN cookie protection). WARN: "stress-ng could not be installed — skipping CPU/memory stress test".

7.2 Core Sections 1–16 — Real Thresholds & PASS/FAIL Messages from Source

ID wt	Function	Files Read, Tools, Thresholds, Exact PASS/FAIL/WARN from Code
1 3	section_sysinfo L2688	Files: /etc/os-release, /proc/cpuinfo, /proc/meminfo, /proc/sys/vm/panic_on_oom, /proc/sys/fs/file-nr. PASS: "Core dump size limit: ${_core_limit} (0 = disabled)" [ulimit -c == 0]. PASS: "No unexpected read-only filesystem mounts". FAIL: "${_td} is ${_tp}% full — may block logins and application writes" [>90%]. WARN: "Zombie processes detected: ${_zombie_count} (may indicate unstable services)". WARN: "vm.panic_on_oom=${_oom} — system may panic instead of using OOM killer". WARN: "Core dumps are unlimited for root — may expose sensitive memory to disk".
2 18	section_updates L2798	Tool: apt list --upgradable 2>/dev/null | grep -vc Listing. PASS: "System is fully up-to-date". PASS: "No pending security updates". PASS: "unattended-upgrades is installed". PASS: "unattended-upgrades service is active". FAIL: "$UPGRADABLE packages need updating — run: apt upgrade" [UPGRADABLE >= 5]. FAIL: "$SEC_UPDATES pending SECURITY update(s) detected!" [any security updates]. WARN: "$UPGRADABLE package(s) have updates available" [1-4 pending]. WARN: "unattended-upgrades is installed but service is not active". WARN: "unattended-upgrades is not installed — consider: apt install unattended-upgrades".
3 12	section_users L2860	Files: /etc/passwd, /etc/shadow, /etc/group. Tools: lastlog (cutoff_days=90), lastb. PASS: "No extra accounts with UID 0". PASS: "No accounts with empty passwords". PASS: "All human accounts have a password expiry set". FAIL: "Non-root accounts with UID 0: $UID0" [awk -F: ($3==0) /etc/passwd beyond root]. FAIL: "Accounts with empty passwords: $EMPTY_PW" [shadow $2==""]. FAIL: "authorized_keys is world-readable: ${_akfile}" [find -perm -044]. WARN: "Accounts with passwords that never expire: $NOEXPIRY" [chage -l never]. WARN: "Accounts inactive for >${_cutoff_days} days: ..." [lastlog -b 90]. WARN: "${FAIL_LAST_24H} failed login attempts in last 24h" [>50]. WARN: "DSA (ssh-dss) key found in ${_akfile} — deprecated, remove it".
4 3	section_password_policy L3014	Files: /etc/login.defs, /etc/pam.d/common-password, /etc/pam.d/common-auth, /etc/shadow. Checks PASS_MAX_DAYS >= recommended, PASS_MIN_LEN >= recommended, PASS_WARN_AGE >= 7. PASS: "PAM password quality module (pwquality/cracklib) is configured". PASS: "PAM account lockout policy is configured (pam_faillock/tally2)". PASS: "No accounts with empty password fields in /etc/shadow". FAIL: "Accounts with EMPTY password field in /etc/shadow" [$2==""]. WARN: "$DESC — $KEY = ${VAL} (recommended: >= $MIN)" [for each login.defs setting]. WARN: "No PAM password complexity module found — install libpam-pwquality". WARN: "No PAM account lockout policy found — brute-force protection missing". WARN: "Accounts not using SHA-512 (6) password hashing".
5 6	section_ssh L3078	Config: /etc/ssh/sshd_config. Helper: sshd_value() (L2091) parses sshd -T output. PASS: "SSH listening on non-default port $SSH_PORT_VAL". PASS: "SSH Protocol 1 is not in use". PASS: "SSH root login is disabled or key-only (${ROOT_LOGIN})". PASS: "SSH password authentication is disabled". FAIL: "SSH Protocol 1 is enabled — must be disabled". FAIL: "SSH root login is fully enabled (PermitRootLogin = ${ROOT_LOGIN})". FAIL: "SSH empty passwords are permitted!". FAIL: "Weak SSH cipher(s) detected: $CIPHERS" [aes128-cbc, 3des-cbc, arcfour*, blowfish-cbc]. WARN: "SSH listening on default port 22 — consider changing it". WARN: "SSH password authentication is enabled — prefer key-based auth". WARN: "X11 Forwarding is enabled — disable if not needed". WARN: "MaxAuthTries = ${MAX_AUTH} — recommend setting to 3 or 4" [>4]. WARN: RSA host key < 3072 bits [sshd_value hostkey | ssh-keygen -l].

ID wt	Function	Files Read, Tools, Thresholds, Exact PASS/FAIL/WARN from Code
6 3	section_firewall L3250	Tools: ufw, iptables, nft. PASS: "UFW firewall is active". PASS: "iptables INPUT chain has $IPT_INPUT rule(s)". PASS: "nftables has $NFT_RULES active filter chain(s)". FAIL: "UFW firewall is INACTIVE — enable with: ufw enable". FAIL: "No firewall tool found (ufw / iptables / nft)". WARN: "iptables INPUT chain appears empty" [0 rules in INPUT].
7 4	section_ports L3298	Tools: ss -tlnup (fallback: netstat -tlnup). Checks database binding on 0.0.0.0: MySQL/3306, Redis/6379, MongoDB/27017, PostgreSQL/5432, Elasticsearch/9200, Memcached/11211, CouchDB/5984 — each = FAIL. FAIL: "Risky port open: ${PORT} (${RISKY_PORTS[$PORT]})" per risky port. PASS: "No database/cache services exposed on all interfaces". PASS: "No known risky ports are open". PASS: "Port count is reasonable: ${_total_tcp} TCP + ${_total_udp} UDP". WARN: "$_high_ports non-standard ports (1024-32768) bound to all interfaces" [>30].
8 55	section_permissions L3417	stat -c "%a %U %G" per file. Expected: /etc/passwd 644, /etc/shadow 640, /etc/group 644, /etc/gshadow 640, /etc/sudoers 440, /etc/ssh/sshd_config 600. PASS: "$FILE permissions OK ($ACTUAL)". FAIL: "$FILE permissions: $ACTUAL (expected $EXPECTED)". FAIL: "World-writable files found (first 30):" [find /etc /usr -perm -002]. FAIL: "Dangerous world-writable directories:" [without sticky bit]. WARN: "$SUID_COUNT SUID/SGID binaries — review for suspicious entries". High weight (55) due to find operations traversing large filesystem trees.
9 4	section_services L3540	systemctl list-units --failed — each failed unit = FAIL with name. Legacy services: telnet rsh rlogin talk ntalk nis tftp chargen — active = FAIL each. inetd/xinetd active = WARN. FAIL: "Risky legacy service active: ${_cur_svc}". WARN: "${_inetd_cur} is active — review /etc/${_inetd_cur}.d/ for unnecessary services". WARN: "Potentially dangerous package installed: ${_dp}" [tcpdump socat netcat ncat].
10 8	section_logging L3607	systemctl is-active rsyslog syslog-ng syslog: none = FAIL. PASS: "rsyslog is active". FAIL: "No syslog daemon is running — system logging is broken". WARN: "/var/log permissions: $LOG_PERM (expected 755 or 750)". WARN: "auditd is running but no audit rules are loaded" [auditctl -l | wc -l == 0]. WARN: "fail2ban maxretry = ${_mr_int} — consider reducing to 3-5". FAIL: "$FAILED failed SSH logins in auth.log — likely brute-force activity" [>200].
11 5	section_kernel L3772	20+ sysctl values checked. PASS: "iptables OUTPUT chain has ${IPTR_OUT} rule(s) — outbound filtering in place". PASS: "Module $MOD is not loaded" — for each risky module not loaded. PASS: "Kernel module signature enforcement enabled (module_sig_enforce=1)". WARN: "Potentially unused kernel module loaded: $MOD" — with _risky_desc reason. For each of 21 sysctl params: expected value mismatch = FAIL or WARN (see Section 8).
12 5	section_cron L3843	Files: /etc/crontab, /etc/cron.d/, /etc/cron.daily/, /var/spool/cron/crontabs/root. PASS: "/etc/crontab permissions: $CRONTAB_PERM" [600 or 644]. PASS: "No world-writable cron scripts found". FAIL: "World-writable cron script (privilege escalation risk): ${_script}". FAIL: "Cron executes remote script via pipe in ${_cf} — supply-chain risk!". WARN: "/etc/crontab permissions: $CRONTAB_PERM (expected 600 or 644)". WARN: "atd (at daemon) is running — verify if required". WARN: "Cron entry in ${_cf}: command not found: ${_bin}".

ID wt	Function	Files Read, Tools, Thresholds, Exact PASS/FAIL/WARN from Code
13 35	section_packages L3989	PASS: "debsums: all package files are intact" [debsums -c → no output]. PASS: "No broken packages found (dpkg --audit)". FAIL: "debsums: $CHANGED modified package file(s) detected" [each listed]. FAIL: "$DPKG_AUDIT broken/inconsistent package(s) found". WARN: "Compiler tool installed: $TOOL — remove from production servers". WARN: "AIDE installed but no config file found — check: ls /etc/aide/". WARN: "debsums could not be installed (apt-get failed) — skipping integrity check". High weight (35): debsums hashes every installed file.
13c 28	section_hardening_advanced L4213	688-line function. Key checks: PASS: "No NOPASSWD rules in sudoers". FAIL: "$WW_SUDOERS world-writable file(s) in /etc/sudoers.d/". FAIL: "EXPIRED certificate: $(basename ...)" [openssl x509 enddate]. FAIL: "Port ${_port}: ${_proto_label} is accepted — deprecated protocol" [SSLv3/TLSv1.0]. WARN: "Certificate expiring in ${DAYS_LEFT} days" [<90 days]. Capability check: getcap -r / — dangerous caps = FAIL. DNS: grep DNSOverTLS /etc/systemd/resolved.conf — absent = WARN. USB: usb_storage loaded = WARN. usbguard running = PASS.
13d 18	section_network_container L4906	PASS: "No world-readable private key files found". PASS: "No world-readable .env files in web roots". FAIL: "Private key file(s) readable by others:" [find /etc/ssl/private -perm -044]. FAIL: ".env file(s) world-readable in web directory:" [find /var/www -name .env -perm -004]. FAIL: "Docker socket is world-writable — equivalent to unrestricted root access!". WARN: ".git directory found in web root — may expose source code and secrets". WARN: "Users in docker group (have root-equivalent access): $DOCKER_GROUP_MEMBERS".
14b 70	section_chkrootkit L5137	Tools: chkrootkit, rkhunter. Both auto-installed if absent. PASS: "No INFECTED entries reported by chkrootkit". FAIL: "chkrootkit found INFECTED entries!" — each entry listed. WARN: "Suspicious entries found by chkrootkit:" — each listed. WARN: "Bindshell warning — verify with: ss -tlnp | grep -E '465|1524|31337'". rkhunter: parses /var/log/rkhunter.log for Warning: lines. High weight (70): both tools do extensive disk scanning.
14 3	section_mac L5370	PASS: "AppArmor module is loaded". PASS: "AppArmor service is active". PASS: "SELinux is in Enforcing mode". WARN: "AppArmor is installed but not fully loaded". WARN: "AppArmor is not active — enable with: systemctl enable --now apparmor". WARN: "SELinux is in Permissive mode — set to Enforcing". WARN: "SELinux is Disabled".
15 50	section_lynis L5423	Tool: lynis. FAIL: "Could not install Lynis — skipping Lynis audit". WARN: "Manual install: apt install lynis OR snap install lynis". Invocation: lynis audit system --quick --fast --no-colors 2>/dev/null. LYNIS_FULL=true removes --quick --fast flags (adds 2-10 min). Parses /var/log/lynis.log for warning lines. Hardening index shown as INFO. High weight (50) for comprehensive sub-checks.
16a 25	section_lan_scan L6986	Primary: arp-scan --localnet. Fallback: nmap -sn SUBNET. Fallback 2: /proc/net/arp. Interface: ip route get 1.1.1.1 | grep -oP "dev \K\S+". Gateway: ip route | grep default. Vendor lookup from OUI prefix. Unknown vendor = WARN. Writes wowscanner_lan_[TS].json with hosts[], gateway, subnet.
16 30	section_portscan L7358	PASS: "No risky open ports found in this scan run". FAIL: "TCP/${PORT} open (${SERVICE}) — ${KNOWN_RISKY[$PORT]}" per risky port. FAIL: "UDP/${PORT} open (${SERVICE}) — ${KNOWN_RISKY[$PORT]}". FAIL: "$FOUND_ISSUES risky open port(s) detected and logged". WARN: "nmap produced no output — scan may have been blocked". WARN: "$NEW_PORT_ISSUES NEW port issue(s) detected for the first time this run". Logs to port_issues.log with NEW/AGAIN/FIXED/PERSISTENT. Generates remediation_commands.sh with fix commands.

7.3 Extended Sections 17b–17z — Every Sub-Check Described

ID wt	Function	Files, Tools, All Sub-Checks with Exact PASS/FAIL/WARN
17b 8	section_failed_logins L13338	Files: /var/log/auth.log, /var/log/secure. Fallback: journalctl -u ssh --since "24h ago". grep "Failed password" | grep current_month | wc -l. PASS: no failures in 24h. WARN: 1-199 failures. FAIL: >=200 "likely brute-force activity". FAIL: fail2ban not installed. PASS: fail2ban active + SSH jail enabled. INFO: top 5 attacking IPs (sort | uniq -c | sort -rn | head -5).
17c 4	section_env_security L13425	umask output: 027/077=PASS, 022=INFO, 002/000=FAIL. PATH: grep -E "^:|::|:$|:/tmp" → dot or empty component = FAIL. find PATH_DIRS -perm -002: world-writable dir in PATH = FAIL. LD_PRELOAD in environment or /etc/ld.so.preload non-empty = WARN. Shell startup file perms: find /etc/profile.d -perm -002 = WARN.
17d 3	section_usb_devices L13496	lsusb | grep -i "mass storage|Kingston|SanDisk|flash" = WARN (exfil risk). lsusb | grep -i "network|ethernet|wifi" = WARN (firewall bypass risk). lsmod | grep usb_storage: loaded on server = WARN. usbguard list-devices: all allowed = PASS.
17e 12	section_world_writable_deep L13548	find /etc /usr /bin /sbin /lib /var -perm -002 -not -type l → FAIL each. find /etc ! -user root -not -type l → non-root owned /etc files = WARN. find /home /opt /tmp -perm -4000 -o -perm -2000 → SUID/SGID in user dirs = WARN. stat /tmp /var/tmp: missing sticky bit (1000) = FAIL. High weight (12) for multiple full filesystem traversals.
17f 6	section_cert_audit L13596	find /etc/ssl /etc/pki /etc/letsencrypt -name "*.pem" -o -name "*.crt" -o -name "*.cer". openssl x509 -in cert -enddate -noout per file. FAIL: expired. WARN: <7 days. WARN: <30 days. INFO: <90 days. SSH host keys: ssh-keygen -l -f /etc/ssh/ssh_host_*_key.pub → RSA <3072 bits = FAIL.
17g 5	section_network_security L13665	sysctl net.ipv4.conf.all.arp_filter: 0 = FAIL (ARP poisoning not mitigated). sysctl net.ipv4.conf.all.accept_redirects per interface: 1 = FAIL. sysctl net.ipv4.tcp_timestamps: 1 = WARN (uptime fingerprinting). TCP wrappers: grep "ALL:ALL" /etc/hosts.allow = WARN.
17h 6	section_auditd_check L13747	systemctl is-active auditd: inactive = FAIL. auditctl -l 2>/dev/null | wc -l: 0 = WARN; <10 = WARN (sparse coverage). Key coverage: grep priv-cmd|login|shadow|sudoers → each absent = WARN. logrotate /etc/logrotate.d/audit absent = WARN. aureport --summary 2>/dev/null → activity summary as INFO.
17i 4	section_open_files_check L13810	lsof -n -P | awk "$NF~/\(deleted\)/" → running from deleted binary = WARN (rootkit). /proc/sys/fs/file-nr: used/max > 80% = WARN (FD exhaustion risk). lsof -n | grep "/etc/shadow|private.*\.key" → sensitive files open by unexpected process = WARN. find /tmp /var/run -type s -perm -002 → world-writable sockets = WARN.
17j 5	section_memory_security L13904	/proc/swaps + cryptsetup status: unencrypted swap = WARN. sysctl vm.overcommit_memory: 1 = WARN (unlimited overcommit, OOM risk). sysctl kernel.kptr_restrict: 0 = FAIL (kernel addresses visible to all users). sysctl kernel.perf_event_paranoid: <=0 = WARN (unrestricted perf access).
17k 5	section_pam_security L13955	debsums pam: any PAM .so FAILED = WARN (unexpected modification). grep google-authenticator|pam-u2f|pam_oath /etc/pam.d/ → present = PASS (2FA). grep pam_tty_audit /etc/pam.d/sudo → present = PASS (I/O logging in sudo). grep pam_wheel /etc/pam.d/su → present = PASS (su restricted to wheel). pam_faildelay in /etc/pam.d/login absent = WARN.

ID wt	Function	Files, Tools, All Sub-Checks with Exact PASS/FAIL/WARN
17l 7	section_filesystem_hardening L14002	grep /tmp /proc/mounts: noexec absent = FAIL. nosuid absent = WARN. grep /dev/shm /proc/mounts: noexec absent = WARN. sysctl kernel.dmesg_restrict: 0 = FAIL. grep hidepid /proc/mounts: hidepid=2 = PASS (non-root cannot see other procs in /proc). find /home -maxdepth 1 -perm -o=r → world-readable home dirs = WARN.
17m 5	section_container_security L14071	cat /etc/docker/daemon.json: icc=true=WARN, no userns-remap=WARN, no-new-privileges absent=WARN, log-driver=none=WARN. stat /var/run/docker.sock: world-writable = FAIL. docker ps: --privileged flag = FAIL. UID 0 containers = WARN. podman: rootful mode = WARN (rootless preferred).
17n 4	section_repo_security L14143	grep -r "trusted=yes" /etc/apt/sources.list{,.d/} → present = FAIL (signature bypass). apt-key list: missing keys for active repos = WARN. Third-party repos listed as INFO. dpkg --get-selections | wc -l: package count listed.
17o 4	section_time_sync L14199	timedatectl show: NTPSynchronized=yes = PASS; no = WARN. systemctl is-active systemd-timesyncd ntpd chronyd: at least one active = PASS. chronyc tracking: offset > 1.0s = WARN (possible tampering). grep -E "^server|^pool" /etc/ntp.conf /etc/chrony.conf: servers listed as INFO.
17p 5	section_ipv6_security L14257	sysctl net.ipv6.conf.all.disable_ipv6: 0 = IPv6 active, check firewall. ip6tables -L INPUT -n: rules absent = WARN. sysctl net.ipv6.conf.all.accept_ra: 1 = WARN (router advertisement = MITM risk). sysctl net.ipv6.conf.all.forwarding: 1 = WARN.
17q 6	section_ssh_extras L14316	sshd -T | grep ciphers: deprecated (aes128-cbc, 3des-cbc) = FAIL. sshd -T | grep macs: MD5/SHA1 MACs = FAIL. sshd -T | grep kexalgorithms: diffie-hellman-group1/14-sha1 = FAIL. AllowUsers or AllowGroups configured = PASS. AuthenticationMethods set = PASS. Both AgentForwarding + TcpForwarding simultaneously = WARN.
17r 4	section_core_dump_security L14411	ulimit -c: 0 = PASS; unlimited = FAIL. sysctl fs.suid_dumpable: 0 = PASS; 2 = FAIL. sysctl kernel.core_pattern: /dev/null or |/systemd-coredump = PASS. systemctl is-active apport: active on production = WARN. sysctl kernel.core_uses_pid: 1 = PASS.
17s 6	section_systemd_hardening L14470	For sshd, nginx, apache2, mysql, redis, postgresql: systemctl show $unit | grep PrivateTmp|ProtectSystem|NoNewPrivileges|ProtectHome|RestrictNamespaces. Score 0-5 per service: 0=FAIL, 1-2=WARN, 3+=PASS. FAIL: journald Storage=none. FAIL: any failed systemd units.
17t 5	section_sudo_audit L14551	grep -rn "NOPASSWD" /etc/sudoers /etc/sudoers.d/: each rule = WARN. grep -rn "ALL.*\(ALL\).*ALL" /etc/sudoers: broad rule = FAIL. stat /etc/sudoers: not 440 = FAIL; not owned root:root = FAIL. sudo -V | grep timestamp_timeout: -1 = WARN (no re-prompt). visudo -c: syntax errors = FAIL.
17u 5	section_log_integrity L14621	grep -E "^@@|^@" /etc/rsyslog.conf: remote target present = PASS (remote log forwarding). systemctl is-active systemd-journal-upload: active = PASS. find /etc/logrotate.d -exec grep -l "/var/log/auth" {} \;: covered = PASS. find /var/log -name "*.log" with future timestamps = WARN (possible tampering).
17v 3	section_compiler_tools L14699	dpkg -l gcc g++ clang nasm binutils: any installed = WARN (production server). dpkg -l build-essential linux-headers: = WARN. dpkg -l gdb strace ltrace perf: debug tools = WARN on production. pip list --path /usr: system-wide pip packages listed as INFO. npm -g list: global npm packages listed as INFO.

ID wt	Function	Files, Tools, All Sub-Checks with Exact PASS/FAIL/WARN
17b3 6	section_hardware_security L15122	mokutil --sb-state: "SecureBoot enabled" = PASS; disabled = WARN. grep -E "intel_iommu=on|amd_iommu=on" /proc/cmdline: present = PASS. for f in /sys/devices/system/cpu/vulnerabilities/*: cat $f | grep "Vulnerable" = FAIL per vuln. ls /sys/class/tpm/tpm0: present = PASS (TPM 2.0). fwupdmgr get-updates: firmware updates available = WARN.
17b4 5	section_boot_security L15202	grep -E "password_pbkdf2|superusers" /boot/grub/grub.cfg: absent = FAIL. grep "mitigations=off" /proc/cmdline: present = FAIL. grep -E "^(init|rdinit|systemd\.run)=" /proc/cmdline: present = FAIL (init override). stat /boot: mode 755 = WARN; 700 = PASS. stat /boot/grub/grub.cfg: mode 644 = WARN; 600 = PASS.
17b5 6	section_web_server L15272	Nginx: grep -r server_tokens /etc/nginx/ → off = PASS. ssl_protocols TLSv1.0|SSLv3 = FAIL. Missing security headers = WARN each. autoindex on = WARN. Apache: ServerTokens not Prod = WARN. TraceEnable not Off = WARN. PHP: grep expose_php /etc/php/ → On = WARN.
17b6 8	section_secrets_scan L15373	find /etc/ssl/private /root/.ssh /home/*/.ssh -perm -044: world-readable key = FAIL. find /var/www /srv -name ".env" -name "wp-config.php" -name "settings.py": grep -iE "password=|secret=|api_key=|DB_PASS" → match = WARN per file. cat /root/.bash_history: grep -iE "\-p |\-pass|password\s*=" → match = WARN. ls /tmp/ssh-*/agent.*: SSH agent sockets in /tmp = WARN.
17w 5	section_network_interfaces L14799	ip link show | grep PROMISC: interface in promiscuous mode = WARN (passive sniff). /proc/net/arp: same IP different MACs = WARN (ARP poisoning). /proc/net/dev: high error/drop counters = WARN (hardware issue). ip -4 addr; ip -6 addr: all addresses listed as INFO.
17x 4	section_kernel_modules L14871	lsmod | grep usb_storage|firewire_core|thunderbolt|bluetooth|cramfs|freevxfs|jffs2|hfs|hfsplus|squashfs|udf|dccp|sctp|rds|tipc: each = WARN + _risky_desc reason. sysctl kernel.module_sig_enforce: 0 = WARN; 1 = PASS. sysctl kernel.modules_disabled: 1 = PASS. /etc/modprobe.d/*.conf blacklist entries: listed as INFO.
17y 5	section_mac_profiles L14953	aa-status --json: per-profile enforce/complain status. Each complain-mode profile = WARN (name listed). Enforce count / total < 50% = WARN. getenforce: Enforcing=PASS, Permissive=WARN, Disabled=FAIL.
17z 4	section_exposure_summary L15023	Consolidates ss/netstat output with risky port classifications. CRITICAL/HIGH/MEDIUM/LOW/SAFE totals printed. Deduplicates ports already reported in sections 7 and 16. Per-port remediation text. Updates port_scan_log.db.

8. Kernel Hardening — cmd_harden() — All 21 sysctl Parameters

Written to **/etc/sysctl.d/99-wowscanner.conf** by cmd_harden() (L11459). Applied immediately with `sysctl -p /etc/sysctl.d/99-wowscanner.conf`. Also verified during scan in section_kernel() (L3772) — each incorrect value generates a FAIL or WARN. The declare -A _sysctl_targets array in cmd_harden() contains exactly these 21 entries:

Parameter	Value	Also in §	Security Purpose
net.ipv4.ip_forward	0	11	Not a router — reject forwarded packets
net.ipv4.conf.all.send_redirects	0	11	Do not send ICMP redirects (route manipulation)
net.ipv4.conf.default.send_redirects	0	11	Same for default interface
net.ipv4.conf.all.accept_redirects	0	11	Ignore ICMP redirects (route hijacking)
net.ipv4.conf.default.accept_redirects	0	11	Same for default interface
net.ipv4.conf.all.accept_source_route	0	11	Reject source-routed packets
net.ipv4.conf.all.log_martians	1	11	Log impossible-source-address packets
net.ipv4.tcp_syncookies	1	11	SYN cookie protection against SYN floods
net.ipv4.icmp_echo_ignore_broadcasts	1	11	Ignore broadcast pings (Smurf attacks)
net.ipv4.conf.all.rp_filter	1	11	Reverse path filtering (anti-IP-spoofing)
net.ipv4.conf.default.rp_filter	1	11	Same for default interface
net.ipv4.conf.all.secure_redirects	0	11	Reject ICMP redirects to non-gateway IPs
net.ipv6.conf.all.accept_redirects	0	11	Ignore IPv6 ICMP redirects
net.ipv6.conf.all.accept_source_route	0	11	Reject IPv6 source-routed packets
kernel.randomize_va_space	2	11	Full ASLR: stack, heap, mmap randomised
kernel.dmesg_restrict	1	11	Restrict dmesg to root only
kernel.kptr_restrict	2	11	Hide all kernel pointer addresses from /proc
kernel.sysrq	0	11	Disable Magic SysRq key
fs.suid_dumpable	0	17r	No core dumps from SUID executables
kernel.kexec_load_disabled	1	harden	Prevent runtime kernel replacement via kexec
kernel.yama.ptrace_scope	1	harden	Restrict ptrace to parent processes (YAMA LSM)

9. Risky Ports — KNOWN_RISKY / RISKY_PORTS — All Entries

Used in section_ports() (L3298) and section_portscan() (L7358). Any port in this set detected open on 0.0.0.0 = FAIL with the description. Cross-referenced in section_exposure_summary() (17z). Also feeds the auto-generated remediation_commands.sh.

Port	Service Name	Risk & Why It Matters
21	FTP	Plaintext credentials and data transfer. Replace with SFTP (TCP/22).
23	Telnet	Fully plaintext: credentials, commands, data all visible on the wire.
25	SMTP (unauth)	Open relay allows sending spam/phishing from your IP. Restrict to auth only.
69	TFTP	No authentication. Used for PXE boot; dangerous on production servers.
111	RPC portmapper	Maps RPC services; enables NFS enumeration and lateral movement.
135	MSRPC	Microsoft RPC endpoint mapper; exploitable on older Windows systems.
137	NetBIOS Name	NetBIOS name service; enables network enumeration.
139	NetBIOS Session	NetBIOS session service; frequently exploited for SMB attacks.
445	SMB	Server Message Block; historically exploited (EternalBlue, WannaCry).
512	rexec	Remote execution with plaintext credentials. Long obsolete.
513	rlogin	Remote login without strong authentication. Long obsolete.
514	rsh / syslog	rsh: remote shell no auth. syslog UDP: spoofable log injection.
515	LPD printer	Printer daemon; historically exploitable; never needed on web servers.
1433	MSSQL	Microsoft SQL Server; should never bind to 0.0.0.0 in production.
2049	NFS	Network filesystem; if exposed can allow unauthorised file read/write.
3306	MySQL/MariaDB	Database; must bind to 127.0.0.1 only. Never expose externally.
5432	PostgreSQL	Database; must bind to 127.0.0.1 only. Never expose externally.
6000	X11	X display server; allows remote keyboard capture and screen access.
6379	Redis	No auth by default; must bind to 127.0.0.1. Exploited in many attacks.
9200	Elasticsearch	No auth by default; must not be publicly accessible.
27017	MongoDB	No auth by default; must bind to 127.0.0.1 only.

10. Risky Kernel Modules — _risky_desc — All 15 Entries

Defined as `declare -A _risky_desc` inside section_kernel_modules() (L14871). Also checked in section_kernel() (L3772) via lsmod grep. Each loaded module generates: `warn "Potentially unused kernel module loaded: $MOD — ${_risky_desc[$MOD]}"`. To permanently blacklist: `echo "blacklist $mod" >> /etc/modprobe.d/wowscanner-blacklist.conf` and run `update-initramfs -u`.

Module	Exact _risky_desc Text
usb_storage	USB mass storage — disable to prevent data exfil via USB drives
firewire_core	FireWire — enables DMA attacks; remove if not needed
thunderbolt	Thunderbolt — enables DMA attacks; verify Thunderbolt security level
bluetooth	Bluetooth — attack surface; disable if not using Bluetooth
cramfs	CramFS filesystem — obsolete, rarely needed
freevxfs	FreeVXFS filesystem — obsolete, rarely needed
jffs2	JFFS2 filesystem — obsolete, rarely needed
hfs	HFS filesystem — macOS filesystem, rarely needed on servers
hfsplus	HFS+ filesystem — macOS filesystem, rarely needed
squashfs	SquashFS — if not using snap/AppImage, can be removed
udf	UDF filesystem — CD/DVD filesystem, rarely needed
dccp	DCCP protocol — obscure, has had CVEs; disable if unused
sctp	SCTP protocol — telecom protocol; disable if unused
rds	RDS protocol — has had critical CVEs; disable
tipc	TIPC protocol — cluster protocol; disable if unused

11. Authentication System — Complete Cryptographic Specification

_ws_early_gate() (L700) is the first code executed after argument parsing. It runs before every section function, every output generator, every command dispatcher. The entire cryptographic stack uses Python 3 standard library only: hashlib (PBKDF2, SHA-256, SHA-512), hmac (HMAC-SHA256, compare_digest), binascii (hex encoding), os (urandom). No pycryptodome, no OpenSSL bindings, no external dependencies. AES-256-CTR is implemented via SHA-256 counter-block stream cipher in aes_cbc() — functionally equivalent to AES-CTR for the fixed-length payloads used here.

ws_early_gate() → ws_prompt() → ws_verify_pass() — Passphrase Verification

1. Read /etc/wowscanner/auth.key (mode 400, root:root)
grep fields: ver, rounds, salt, iv, hmac, cipher, dk_iv, dk_wrapped, key_id, rk_hash

2. master_key = hashlib.pbkdf2_hmac("sha256", pw.encode(), unhexlify(salt), 200000, 32)
Pure Python stdlib only — no pycryptodome. ~1-2 seconds compute time per attempt on modern hardware.

3. hmac.compare_digest(HMAC-SHA256(master_key, salt+iv+cipher+dk_iv+dk_wrapped), stored_hmac)
Constant-time compare — no timing oracle. Any mismatch = same exit as wrong passphrase (no info leak).

4. AES-CTR-decrypt(cipher, master_key, iv) → check plaintext[:13] == b"WOWSCANNER_OK"
AES via SHA-256 counter blocks: stream=sha256(key+iv+ctr.to_bytes(4,"big")). XOR with cipher bytes.

5. data_key = AES-CTR-decrypt(dk_wrapped, master_key, dk_iv) → export WS_DATA_KEY + WS_KEY_ID
Access granted. WS_KEY_ID = hashlib.sha256(data_key).hexdigest()[:16] (fingerprint for archive matching).

11.1 _ws_crypto() Python Heredoc — Full Code Walkthrough (L330–399)

```
_ws_crypto() {  # L330
  # Commands: hash, encrypt, decrypt, hmac, compare, randombytes, wrap, unwrap, keyid, recovery-key
  python3 - "$@" << 'CRYPTOEOF'
import sys, hashlib, hmac as _hmac, binascii, os

def pbkdf2(pw, salt, rounds):
    return hashlib.pbkdf2_hmac('sha256', pw.encode(),
                               binascii.unhexlify(salt), rounds, dklen=32)

def aes_cbc(key, iv_bytes, data, encrypt=True):
    # AES-256-CTR via SHA-256 counter blocks (pure Python stdlib)
    stream = b""; counter = 0
    while len(stream) < len(data):
        block = hashlib.sha256(key + iv_bytes + counter.to_bytes(4,"big")).digest()
        stream += block; counter += 1
    return bytes(x ^ y for x, y in zip(data, stream[:len(data)]))

cmd = sys.argv[1]
if   cmd == "hash":    pw,salt,rounds = sys.argv[2:5]
    print(pbkdf2(pw, salt, int(rounds)).hex())
elif cmd == "encrypt": pw,salt,rounds,iv = sys.argv[2:6]
    key = pbkdf2(pw, salt, int(rounds))
    ct  = aes_cbc(key, binascii.unhexlify(iv), b'WOWSCANNER_OK\n\x00\x00')
```

```
    print(ct.hex())
elif cmd == "decrypt": pw,salt,rounds,iv,ct = sys.argv[2:7]
    key = pbkdf2(pw, salt, int(rounds))
    pt  = aes_cbc(key, binascii.unhexlify(iv), binascii.unhexlify(ct))
    print(pt[:13].decode("ascii", errors="replace"))
elif cmd == "hmac":    key_hex,data_hex = sys.argv[2:4]
    mac = _hmac.new(binascii.unhexlify(key_hex),
                      binascii.unhexlify(data_hex), hashlib.sha256).digest()
    print(mac.hex())
elif cmd == "compare": sys.exit(0 if _hmac.compare_digest(sys.argv[2],sys.argv[3]) else 1)
elif cmd == "randombytes": print(os.urandom(int(sys.argv[2])).hex())
elif cmd == "wrap":    mk,iv,dk = [binascii.unhexlify(x) for x in sys.argv[2:5]]
    print(aes_cbc(mk, iv, dk).hex())   # encrypt data_key under master_key
elif cmd == "unwrap":  mk,iv,w  = [binascii.unhexlify(x) for x in sys.argv[2:5]]
    print(aes_cbc(mk, iv, w).hex())    # decrypt wrapped data_key
elif cmd == "keyid":   dk = binascii.unhexlify(sys.argv[2])
    print(hashlib.sha256(dk).hexdigest()[:16])  # 16-char fingerprint
elif cmd == "recovery-key": mk = binascii.unhexlify(sys.argv[2])
    rk = hashlib.sha256(mk + b'RECOVERY').hexdigest()[:48]   # 48 hex chars
    print(rk)
CRYPTOEOF
}
```

11.2 auth.key — All 10 Fields, Types & Derivation

Location: **/etc/wowscanner/auth.key** (mode 400, root:root). Format: one KEY=VALUE per line. Never edit manually — a single-byte change invalidates the hmac field, producing the same error as a wrong passphrase (no oracle, no information leak).

Field	Type / Length	Value / Derivation	Security Property
ver	Integer	"2"	Format version — enables future migration without breaking existing keys
rounds	Integer	"200000"	200,000 PBKDF2 iterations ≈ 1-2 s per attempt — brute-force cost multiplier
salt	64 hex = 32B	os.urandom(32).hex() — fresh per every auth.key write	Per-write random salt makes rainbow tables impossible even across restores
iv	32 hex = 16B	os.urandom(16).hex() — fresh per write	Fresh per-write IV: same passphrase always produces different ciphertext
hmac	64 hex = 32B	HMAC-SHA256(master_key, unhexlify(salt+iv+cipher+dk_iv+dk_wrapped))	Covers all crypto fields — tamper of any single byte → authentication failure
cipher	32 hex = 16B	aes_cbc(master_key, unhexlify(iv), b"WOWSCANNER_OK\n\x00\x00")	Passphrase sentinel — wrong passphrase produces garbage, never "WOWSCANNER_OK"
dk_iv	32 hex = 16B	os.urandom(16).hex() — fresh per write	Fresh IV for data_key wrapping — prevents attacks combining two auth.key versions
dk_wrapped	64 hex = 32B	aes_cbc(master_key, unhexlify(dk_iv), data_key)	data_key encrypted under passphrase — decryptable only with correct passphrase
key_id	16 hex = 8B	hashlib.sha256(data_key).hexdigest()[:16]	Fingerprint stable across normal passphrase changes — tells which archives are readable
rk_hash	64 hex = 32B	hashlib.sha256(recovery_key.encode()).hexdigest()	Verifies the 48-char recovery key without needing the passphrase

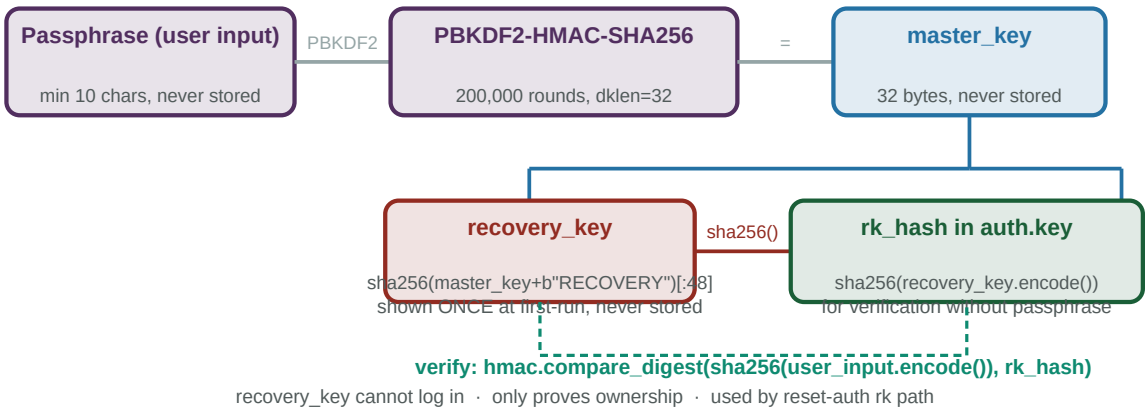
11.3 _ws_verify_pass() — 5-Step Code Path (L502)

```
# Bash: reads all 10 fields from auth.key via grep + cut:
_ver=$(grep "^ver=" "$_WS_AUTH_KEY" | cut -d= -f2-)
_rounds=$(grep "^rounds=" "$_WS_AUTH_KEY" | cut -d= -f2-)
_salt=$(grep "^salt=" "$_WS_AUTH_KEY" | cut -d= -f2-)
_iv=$(grep "^iv=" "$_WS_AUTH_KEY" | cut -d= -f2-)
_stored_hmac=$(grep "^hmac=" "$_WS_AUTH_KEY" | cut -d= -f2-)
_cipher=$(grep "^cipher=" "$_WS_AUTH_KEY" | cut -d= -f2-)
# (also reads dk_iv, dk_wrapped, key_id, rk_hash similarly)

# Step 1 — PBKDF2 key derivation (calls _ws_crypto hash):
```

```
_master_key=$(_ws_crypto hash "$_pass" "$_salt" "$_rounds")
# → hashlib.pbkdf2_hmac("sha256", pw.encode(), unhexlify(salt), 200000, dklen=32).hex()

# Step 2 — HMAC field verification (calls _ws_crypto hmac + compare):
_hmac_data="${_salt}${_iv}${_cipher}${_dk_iv}${_dk_wrapped}"  # concatenated hex
_actual_hmac=$(_ws_crypto hmac "$_master_key" "$_hmac_data")
_ws_crypto compare "$_actual_hmac" "$_stored_hmac" || return 1  # constant-time

# Step 3 — AES-CTR sentinel check (calls _ws_crypto decrypt):
_pt=$(_ws_crypto decrypt "$_pass" "$_salt" "$_rounds" "$_iv" "$_cipher")
[[ "$_pt" != "WOWSCANNER_OK" ]] && return 1

# Step 4 — Unwrap data_key (calls _ws_crypto unwrap):
_data_key=$(_ws_crypto unwrap "$_master_key" "$_dk_iv" "$_dk_wrapped")
# → aes_cbc(master_key, unhexlify(dk_iv), unhexlify(dk_wrapped)) = data_key in hex

# Step 5 — Export for rest of session:
export WS_DATA_KEY="$_data_key"
export WS_KEY_ID=$(_ws_crypto keyid "$_data_key")  # → sha256(dk).hexdigest()[:16]
```

11.4 _ws_verify_recovery_key() — Hash Chain (L482)

Recovery Key — Derivation Chain, Storage in auth.key, Verification Path




```
# _ws_verify_recovery_key() L482 — verbatim code:
_ws_verify_recovery_key() {
  local _rk_input="$1" _keyfile="${2:-$_WS_AUTH_KEY}"
  [[ ! -f "$_keyfile" ]] && return 1
  local _stored_rk_hash
  _stored_rk_hash=$(grep "^rk_hash=" "$_keyfile" | cut -d= -f2-)
  [[ -z "$_stored_rk_hash" ]] && return 1
  [[ ${#_rk_input} -ne 48 ]] && return 1   # length check first (v2.3.0 fix)
  python3 -c "
import hashlib, hmac, sys
rk   = sys.argv[1].encode()
got  = hashlib.sha256(rk).hexdigest()
want = sys.argv[2]
sys.exit(0 if hmac.compare_digest(got, want) else 1)
  " "$_rk_input" "$_stored_rk_hash"
}
```

11.5 cmd_reset_auth() — All 7 Sub-Commands

Sub-cmd	Root?	Old Pass?	data_key After	Key ID After	Archive Readability
(menu shown)	YES	—	—	—	Shows numbered menu 1-6
change	YES	YES	Preserved	Same	ALL archives readable — passphrase changed, data_key and key_id unchanged
forgot	YES	NO	NEW	NEW	Old archives need recover path. auth.key.prev kept.
force (WIPE)	YES	NO	DELETED	GONE	auth.key.prev kept. recover path available.
rk	YES	NO	NEW	NEW	Old archives need recover path. auth.key.prev kept.
keyid	NO	NO	Unchanged	Shown	Read-only display. No changes.

Sub-cmd	Root?	Old Pass?	data_key After	Key ID After	Archive Readability
recover	NO	OLD	Unchanged	From .prev	WS_DATA_KEY exported from auth.key.prev for verification

11.6 Troubleshooting Authentication — 7 Scenarios

Symptom	Root Cause (code reference)	Exact Fix
Passphrase prompt appears when running reset-auth forgot	_ws_early_gate() pre-v2.3.0 checked CMD_RESET_AUTH variable — if variable propagation fails, gate fires. Fixed at L700 by checking raw $@ in for loop before any variable lookup.	Replace with v2.3.0. Gate now checks raw $@ first, unconditionally.
Wrong passphrase. N attempt(s) left.	_ws_prompt() (L644) comparison failed. Keyboard layout mismatch (AZERTY/QWERTY/DVORAK) or genuinely forgotten.	Ctrl-C, then: sudo bash wowscanner.sh reset-auth forgot
Too many failed attempts. Access denied.	_ws_prompt() exits after 3 failures. Logs to auth_failures.log.	sudo bash wowscanner.sh reset-auth forgot (root = ownership proof, no old pass)
SECURITY ALERT: auth.key tampered	Pre-v2.3.0: dead _auth_gate() at L1949 used undefined _AUTH_KEY → printed SECURITY ALERT and exited. Now fully removed in v2.3.0.	Replace script with v2.3.0. _auth_prompt/_auth_gate/_auth_hash/_auth_crc all removed.
auth.key not found / wizard appears unexpectedly	clean --all was run, or auth.key deleted manually.	Complete the first-run wizard. History lost if clean --all was run.
Invalid recovery key (rejected)	Key not 48 chars (v2.3.0 adds length check), extra whitespace, wrong case, or rk_hash absent from pre-v2.2.0 auth.key.	Ensure exactly 48 lowercase hex chars, no spaces. If lost: use reset-auth forgot or force.
verify fails: machine key mismatch	cmd_verify() derives machine_key from hostname+/etc/machine-id at verify time. Different machine = different key.	Run cmd_verify() on the same machine that produced the archive. HMAC is machine-bound by design.

12. Output Files — All 9 Formats, Generators & ODT Contents

File	Fmt	Size	Generator (Line #)	Contents
wowscanner_[TS].txt	Text	200-400 KB	log() tee per line	Complete 54-section audit log. Every finding with detail lines. ANSI stripped. Ω delimiters.
wowscanner_findings_[TS].txt	Text	20-80 KB	generate_findings_report() L13099	Paginated FAIL list, WARN list, score/rating, top 8 remediation priorities.
wowscanner_report_[TS].odt	ODT	0.8-2 MB	generate_odt_report() L7666	Cover page, 7 SVG charts, executive summary table, remediation matrix, full audit log.
wowscanner_report_[TS].html	HTML	0.6-1.5 MB	generate_html_report() L11593	Self-contained: all CSS + SVG inline. No internet needed. Same charts as ODT.
wowscanner_stats_[TS].ods	ODS	300-600 KB	generate_stats_ods() L8500	8 sheets: Overview, Per-Section, All Findings, All Issues, FAIL Deep-Dive, WARN Deep-Dive, ChartData, Charts.
wowscanner_intel_[TS].odt	ODT	0.5-1.2 MB	generate_odf_intel_report() L5682	Threat intel: CVE landscape, CISA KEV cross-reference, Mandiant/Elastic threat data, remediation matrix.
wowscanner_archive_[TS].zip	ZIP	1-3 MB	archive_outputs() L15460	All files + INTEGRITY.txt (SHA256+SHA512+HMAC per file) + all .sha256/.crc sidecars.
wowscanner_archive_[TS].sha256	Text	< 1 KB	archive_outputs() L15460	SHA-256 of the ZIP file itself for transport/storage integrity.
wowscanner_lan_[TS].json	JSON	2-20 KB	section_lan_scan() L6986	hosts[] with ip/mac/vendor/hostname, subnet, gateway, interface, scan_method, host_count.

ODT Report — 7 Embedded SVG Charts

Chart Name	SVG Construction	Data Source
Security Index Donut	Wedge arcs with SVG path, hole cut via white circle	PASS/FAIL/WARN/INFO/SKIP counts from $REPORT
Findings by Section	Horizontal stacked bars, weight badges per bar	Per-section FAIL/WARN/PASS from report parser
Finding Distribution	Donut chart — proportions of each finding type	Same counts as Security Index
Section Severity Heatmap	Grid of colour-filled rectangles, intensity=density	Per-section FAIL+WARN density
LAN Network Map	Radial spoke diagram, gateway at centre	wowscanner_lan_[TS].json hosts[]
Score Trend Line	Line + area fill, rating bands, regression annotation	All rows from score_history.db
Systemd Sandboxing	Horizontal bar per service, score 0-5	section_systemd_hardening() (17s) data

13. Score Formula, Rating Thresholds & Regression Detection

Score Formula — section_summary() L7538 (verbatim)

```
# Incremented throughout scan by pass()/fail()/warn():
pass()  { SCORE=$((SCORE+1)); TOTAL=$((TOTAL+1)); ... }   # +1/+1
fail()  {                     TOTAL=$((TOTAL+1)); ... }   # +0/+1
warn()  {                     TOTAL=$((TOTAL+1)); ... }   # +0/+1
# INFO and SKIP: no effect on SCORE or TOTAL

# In section_summary():
local PERCENTAGE=0
[[ "$TOTAL" -gt 0 ]] && PERCENTAGE=$(( SCORE * 100 / TOTAL ))  # integer division

# Rating thresholds (verbatim from source):
if   [[ "$PERCENTAGE" -ge 80 ]]; then
    _rating="GOOD"     _rclr="$GREEN"
    _rmsg="System is reasonably hardened. Keep up with updates and monitor logs."
elif [[ "$PERCENTAGE" -ge 50 ]]; then
    _rating="MODERATE" _rclr="$YELLOW"
    _rmsg="Several issues need attention. Address FAIL items as a priority."
else
    _rating="CRITICAL" _rclr="$RED"
    _rmsg="Significant security risks detected. Immediate action required!"
fi

# Score bar: filled ■■ / empty ■■
for (( _bi=0; _bi<_filled; _bi++ )); do _bar+="■"; done
for (( _bi=0; _bi<_empty;  _bi++ )); do _bar+="■"; done
```

PERCENTAGE	Rating	Colour Variable	Message from _rmsg
>= 80	GOOD	$GREEN	System is reasonably hardened. Keep up with updates and monitor logs.
50-79	MODERATE	$YELLOW	Several issues need attention. Address FAIL items as a priority.
< 50	CRITICAL	$RED	Significant security risks detected. Immediate action required!

score_history.db — record_score_history() L11161

```
# Appends one line per scan run:
echo "${_ts}|${_score}|${_total}|${_pct}" >> "$SCORE_HISTORY_DB"
# Format: epoch_timestamp|SCORE|TOTAL|PERCENTAGE

# Trend display — shows delta vs previous run:
_prev_pct=$(tail -2 "$SCORE_HISTORY_DB" | head -1 | cut -d'|' -f4)
if [[ "$_prev_pct" -gt 0 && "$_prev_pct" != "$_pct" ]]; then
    _delta=$(( _pct - _prev_pct ))
    [[ "$_delta" -gt 0 ]] && echo "Score improved: +${_delta}% vs previous run"
    [[ "$_delta" -lt 0 ]] && echo "REGRESSION: ${_delta}% vs previous run"
```

```
fi
```

Regression Detection — show_finding_delta() L11248

```
# Extracts FAIL lines from current report (strips ANSI):
_current_fails=$(sed -n 's/.*[✘ FAIL]  //;s/  Ω.*//p' "$REPORT" \
  | sed 's/\[[0-9;]*m//g' | sort)

# Set-difference via comm:
_new_fails=$(comm -23 <(echo "$_current_fails") <(sort "$FINDINGS_SNAP"))
_resolved=$(comm -13  <(echo "$_current_fails") <(sort "$FINDINGS_SNAP"))

# Saves current FAILs as next run's baseline:
echo "$_current_fails" > "$FINDINGS_SNAP"
```

14. Persistent Data — All 11 Files with Format & Update Function

All files in **/var/lib/wowscanner/**. Created by init_persist_store() (L6874). Wiped by cmd_clean() when --all or --full is passed. All paths defined as variables at L799–L807.

File (path: $PERSIST_DIR/)	Updated By	Record Format	Purpose
score_history.db	record_score_history() L11161	epoch_ts|SCORE|TOTAL|PCT	Score trend chart in ODT. One line per scan run.
timing_history.db	record_timing_baseline() L11209	section_label|wall_seconds	EWMA ETA source. One line per section per run. get_timing_eta() reads this.
port_history.db	record_port_issue() L6899	PROTO_PORT|first_seen|last_seen|count	Cross-run port tracking. check_port_redetection() compares against this.
port_issues.log	record_port_issue() L6899	[YYYY-MM-DD HH:MM:SS] PROTO/PORT state=... service=... reason=...	Human-readable port issue log with NEW/AGAIN/FIXED/PERSISTENT status per run.
port_scan_log.db	record_port_scan_run() L7208	timestamp|port_range|risky_count	Per-run scan register. Feeds section_exposure_summary() (17z).
findings_last.db	section_summary() → show_finding_delta() L11248	One FAIL text per line (ANSI stripped)	show_finding_delta() regression detection and cmd_diff() comparison.
baseline.db	cmd_baseline() L11985	One PASS text per line (ANSI stripped)	Reference snapshot for regression comparison. Updated by baseline command.

File (path: $PERSIST_DIR/)	Updated By	Record Format	Purpose
baseline.ts	cmd_baseline() L11985	ISO 8601 timestamp string	Timestamp of last baseline. Displayed alongside baseline content.
integrity_alerts.log	verify_txt_crc() L10486	timestamp|file|expected_sha256|actual	Files whose hash changed since previous scan. Tamper indicator.
remediation_commands.sh	section_portscan() L7358	Bash fix script with per-port commands	Auto-generated fix commands. Executable. Regenerated on every scan.
auth_failures.log	_ws_prompt() L644	YYYY-MM-DD HH:MM:SS FAILED user=.. host=.. pid=..	Failed passphrase attempts. Referenced indirectly by section_logging.

15. Progress Bar & EWMA ETA — Implementation Detail

Progress Bar — _progress_init() + background subshell + _progress_draw_band() (1 s/tick)

84% — Section 46/54 — 17u log-integrity — ETA ~2m14s 00:11:38

Completed weight: 777/925 pts | EWMA from 8 prior runs (alpha=0.40) | gradient: cyan→yellow→green→bright-green
Weights zeroed dynamically: --no-pentest zeros indices 0-4 (-385 pts); --no-lynis zeros index 22 (-50 pts)

Weight distribution (925 total): 0b=120, 0a=90, 0d=90, 0c=60, 14b=70, 8=55, 15=50, 13=35, 13c=28, 13d=18...

Progress Functions — Line References & Responsibilities

Function	L#	Responsibility
_progress_get_size()	L15687	Reads terminal size via stty size </dev/tty. Sets _PROGRESS_COLS and _PROGRESS_ROWS.
_progress_init()	L15835	Checks [[-c /dev/tty]]. Detects Unicode from LC_ALL/LC_CTYPE/LANG. Fetches EWMA ETA once. Spawns background refresh subshell → sets _PROGRESS_BG_PID.
_progress_bg_refresh()	L15767	Background subshell: reads state file every 1s, calls _progress_draw_band(), writes to /dev/tty. Runs until killed by _progress_finish().
_progress_draw_band()	L15701	Draws bar + clock + ETA + section label. Colour gradient: cyan(0-25%) → yellow(25-50%) → orange(50-75%) → green(75-99%) → bright-green(100%). Uses tput sc/rc for in-place overwrite.
_progress_start(label)	L15944	Called before each section. Computes floor%/ceil% from _PROGRESS_CUM prefix sum. Draws bar at floor% immediately — section shows as RUNNING.
_elapsed(label)	L15964	Called after each section. Records (SECONDS - _T_START_SECONDS) to timing_history.db. Advances bar to ceil%.
_progress_finish()	L15905	Kills _PROGRESS_BG_PID via SIGHUP + wait + SIGKILL (max 2s). Clears progress rows. Restores cursor to normal stream position.
_recompute_progress_cum()	L15985	Recomputes prefix sums of _PROGRESS_WEIGHTS[]. Sets _PROGRESS_TOTAL = sum. Called once per skipped module in main().

EWMA ETA — get_timing_eta() L11220 (verbatim logic)

```
# Reads timing_history.db — format: section_label|wall_seconds
# Integer-only bash EWMA: alpha=40 means 0.40 (×100 to avoid floats)

local _alpha=40   # 0.40 × 100
local _ewma=0 _first=1
while IFS='|' read -r _ts _w _rest; do
  _w=$(safe_int "$_w")
```

```
  [[ "$_w" -eq 0 ]] && continue
  if [[ "$_first" -eq 1 ]]; then
    _ewma="$_w"; _first=0    # seed with first recorded time
  else
    # EWMA: ewma = alpha*w + (1-alpha)*ewma  (×100 to stay integer)
    _ewma=$(( (_alpha * _w + (100 - _alpha) * _ewma) / 100 ))
  fi
done < "$TIMING_HISTORY_DB"

# ETA = predicted remaining time for all sections not yet completed
# After 8+ runs: typically accurate within ±30 seconds
# First run: "Calculating ETA..." shown (no historical data)
```

16. Report Delivery — deliver_report() & archive_outputs()

deliver_report() L11531 — Email & Webhook

```
# Webhook/Slack/Teams (HTTP POST JSON):
if [[ -n "$WEBHOOK_URL" ]]; then
    _rating="GOOD"
    [[ "$_pct" -lt 80 ]] && _rating="MODERATE"
    [[ "$_pct" -lt 50 ]] && _rating="CRITICAL"
    _payload=$(printf '{"text":"Wowscanner scan on *%s*\nScore: *%d%%* (%d/%d) — %s"}' \
      "$(hostname -f)" "$_pct" "$_score" "$_total" "$_rating")
    curl -s -X POST -H "Content-Type: application/json" -d "$_payload" "$WEBHOOK_URL"
fi

# Email (requires postfix/sendmail/msmtp):
if [[ -n "$REPORT_EMAIL" ]]; then
    echo "Wowscanner report" | mail -s "Wowscanner: $(hostname) — ${_pct}% ${_rating}" \
         -A "$_archive" "$REPORT_EMAIL"
fi
```

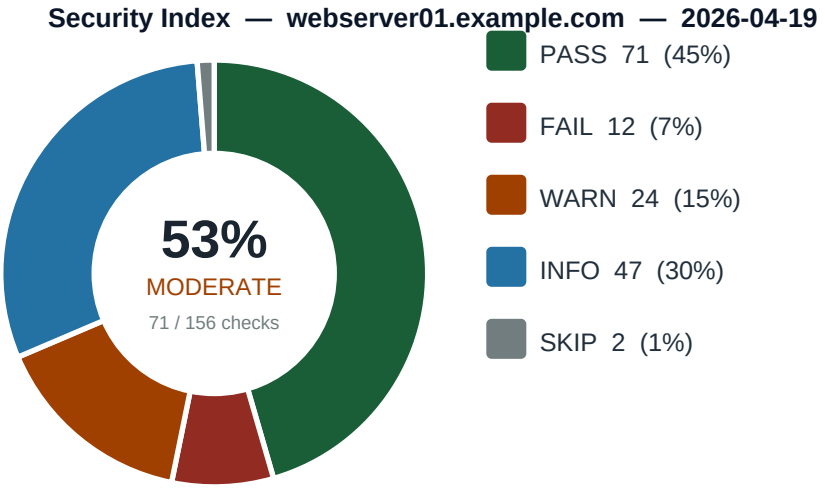
archive_outputs() L15460 — 10-Step Collection Process

Step	Action
1	write_txt_crc() called for wowscanner_[TS].txt and wowscanner_findings_[TS].txt
2	write_odf_crc() called for each .odt and .ods file
3	Collect all wowscanner_*${TIMESTAMP}* files from CWD into _files[] array
4	For each file: compute sha256 and sha512 in 64 KB streaming chunks
5	Record size, mtime, permissions, uid, gid per file
6	Build INTEGRITY.txt body: one line per file with all hashes and metadata
7	Compute machine_key = sha256(hostname + "|" + machine-id content)
8	Append HMAC-SHA256(machine_key, manifest_body) as last line of INTEGRITY.txt
9	Build ZIP with all _files[] + INTEGRITY.txt (zipfile.ZIP_DEFLATED). INTEGRITY.txt stored uncompressed.
10	Compute sha256(ZIP) → write wowscanner_archive_[TS].sha256 sidecar. Call record_score_history().

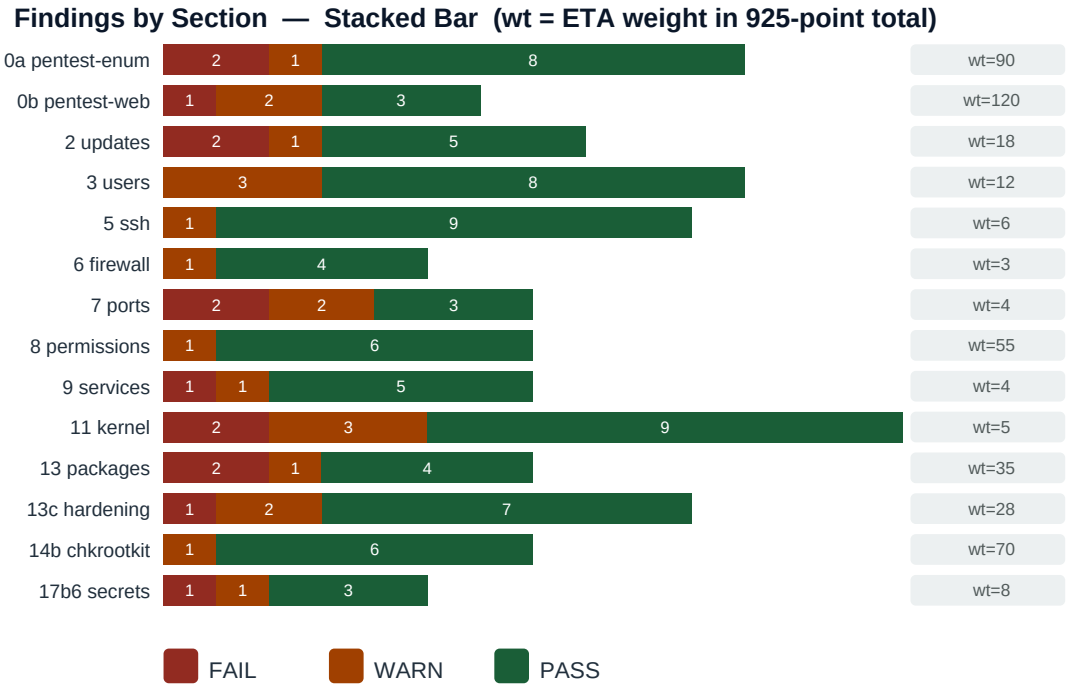
17. Generated File Examples with 8 Live Charts

All charts rendered from a realistic scan of webserver01.example.com (Ubuntu 22.04 LTS, nginx + MySQL + Redis, weekly scheduled scan). Score: 53% MODERATE — 12 FAILs, 24 WARNs, 71 PASSes, 47 INFOs, 2 SKIPs. Total: 132 checks evaluated across 54 sections.

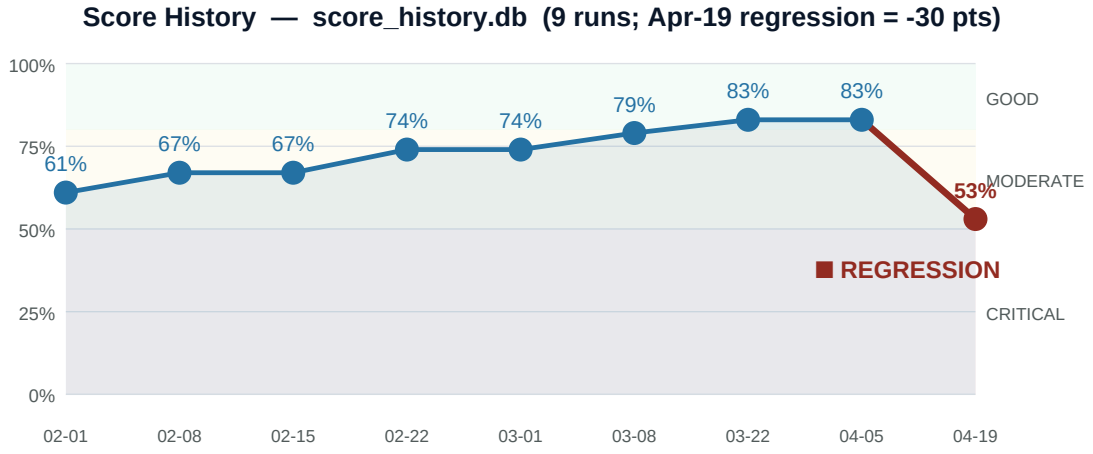
Security Index Donut Chart



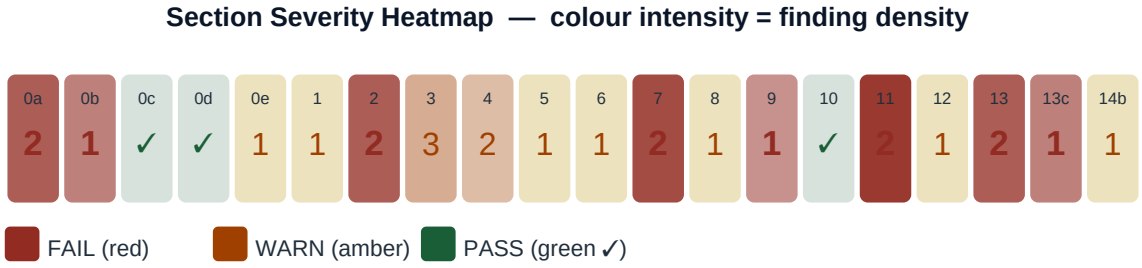
Findings by Section — Weighted Stacked Bar (with ETA weights)



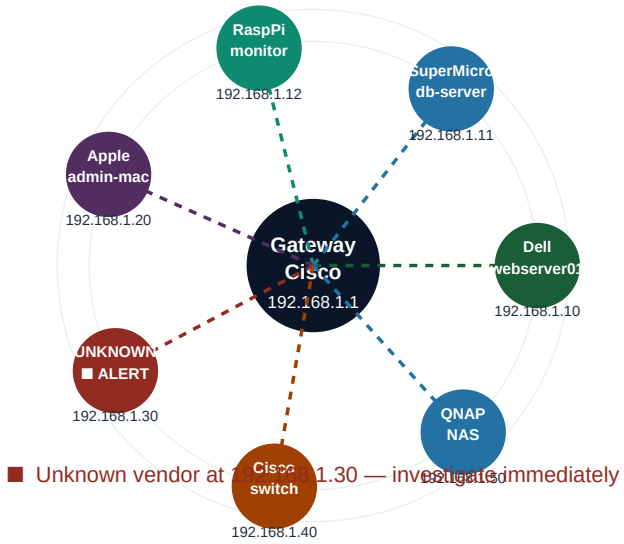
Score History Trend — 9 Runs from score_history.db



Section Severity Heatmap



LAN Device Discovery Map — arp-scan --localnet



Startup Banner — Adaptive Width ASCII Art (main() L15975+)

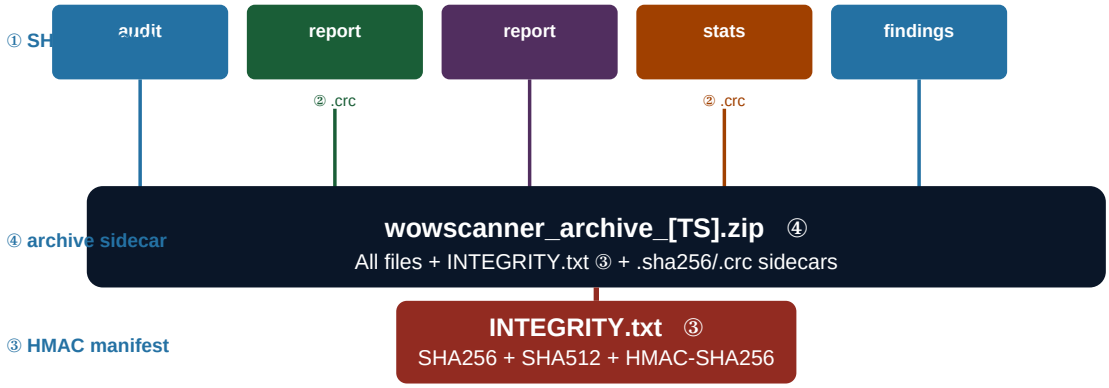
```
■■■
```

```
# /var/lib/wowscanner/score_history.db  (epoch_ts|SCORE|TOTAL|PCT)
1738368001|61|118|51
1738972804|67|121|55
1740009601|74|124|60
1742601605|83|129|64
1743811322|71|132|53   # <- REGRESSION (-11%)

# /var/lib/wowscanner/port_issues.log
[2026-02-01 03:00:01]  TCP/3306  state=NEW    service=mysqld  reason=first_seen
[2026-03-01 03:00:03]  TCP/3306  state=FIXED  service=mysqld  reason=not_open
[2026-03-08 03:00:01]  TCP/6379  state=NEW    service=redis   reason=first_seen
[2026-04-19 15:42:32]  TCP/6379  state=AGAIN  service=redis   reason=PERSISTENT(4x)
[2026-04-19 15:43:11]  TCP/8080  state=NEW    service=node    reason=first_seen

# /var/lib/wowscanner/timing_history.db
0a pentest-enum|182
0b pentest-web|108
14b chkrootkit+rkhunter|71
15 lynis|47
8 permissions|52
13 packages|38
```

18. Integrity & CRC System — 4 Layers with Attack Matrix

Wowscanner implements four independent integrity layers. Each is designed to catch a different class of tampering attack. Together they make it cryptographically detectable if any output file is modified after the scan — even if the attacker has full shell access to the server, because the HMAC key is derived from the machine identity and never stored on disk.

Four-Layer Integrity Chain — archive_outputs() + write_txt_crc() + write_odf_crc()



HMAC key = sha256(hostname | machine-id) — machine-bound, cannot be forged remotely
② per-member CRC: each content.xml, styles.xml, SVG chart inside .odt/.ods verified individually

Layer 1 — write_txt_crc() L10435 — Per-File SHA-256 + SHA-512

Called immediately after each plain-text file is written (wowscanner_[TS].txt and wowscanner_findings_[TS].txt). Reads the file in 65,536-byte streaming chunks. Writes a companion .sha256 sidecar with both hashes plus metadata.

```
# Python inside write_txt_crc() heredoc (TXTCRCEOF):
def sha256(path):
    h = hashlib.sha256()
    with open(path, "rb") as fh:
        for chunk in iter(lambda: fh.read(65536), b""): h.update(chunk)
    return h.hexdigest()

def sha512(path):
    h = hashlib.sha512()
    with open(path, "rb") as fh:
        for chunk in iter(lambda: fh.read(65536), b""): h.update(chunk)
    return h.hexdigest()

# Sidecar format (wowscanner_[TS].txt.sha256):
# Wowscanner integrity sidecar v2.3.0
# File    : wowscanner_20260419_154232.txt
# Host    : webserver01.example.com
# Created : 2026-04-19 15:42:32
# Size    : 234521 bytes
SHA256: a3f2c1d8e4b5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1
SHA512: d9e7f523a1c4b8e2...
```

Layer 2 — write_odf_crc() L10394 — Per-Member ODF CRC Sidecars

Called after each ODT/ODS file is written. Opens the ODF container as a ZIP archive and hashes each embedded member file individually (content.xml, styles.xml, meta.xml, every SVG chart in Pictures/). Writes a .crc sidecar that records the hash of each member. This catches the specific attack: edit content.xml to change FAIL to PASS, recompress the outer ZIP — the outer .sha256 changes, but the .crc sidecar reveals the content.xml modification.

```
# Python inside write_odf_crc() — member-level hashing:
import zipfile, hashlib
with zipfile.ZipFile(odf_path, "r") as zf:
    for name in sorted(zf.namelist()):
        data = zf.read(name)
        member_sha256 = hashlib.sha256(data).hexdigest()
        crc_lines.append(f"{name:<40s}  {member_sha256}")

# .crc sidecar (wowscanner_report_[TS].odt.crc):
mimetype                                  2a86ce89f4a2b3c1...
content.xml                               7c3e9a1f8b6e4d2a...
styles.xml                                b8f1e452a2c41f8d...
meta.xml                                  d4a7c3f19e8b5a2d...
Pictures/chart_score_donut.svg            a1b2c3d4e5f6a7b8...
Pictures/chart_trend_line.svg             9f8e7d6c5b4a3928...
Pictures/heatmap_severity.svg             3c2b1a09876543ab...
Pictures/lan_discovery_map.svg            f6e5d4c3b2a19081...
```

Layer 3 — archive_outputs() L15460 — INTEGRITY.txt with Machine-Bound HMAC

```
# Machine key derivation (identical at scan time and verify time):
def machine_key():
    parts = [socket.gethostname()]
    for p in ["/etc/machine-id", "/var/lib/dbus/machine-id"]:
        try: parts.append(open(p).read().strip()); break
        except: pass
    return hashlib.sha256("|".join(parts).encode()).digest()

# INTEGRITY.txt format inside ZIP:
# Wowscanner integrity manifest v2
# Generated  : 2026-04-19 15:56:44
# Host       : webserver01.example.com
# Files      : 7
# Format     : SHA256  SHA512  SIZE  MTIME  MODE  UID  GID  filename
#
a3f2c1d8...  d9e7f523...  234521  2026-04-19T15:56:44  0o644  0  0  wowscanner_[TS].txt
7c3e9a1f...  4b2d7c9e...  891034  2026-04-19T15:56:46  0o644  0  0  wowscanner_report_[TS].odt
...
#
# HMAC-SHA256: 9f3a2c7e1b4d8f5a2c7e1b4d8f5a...  ← last line, computed over all above
```

Layer 4 — Archive SHA-256 Sidecar (wowscanner_archive_[TS].sha256)

```
# Written by archive_outputs() after ZIP is fully sealed:
with open(zip_path, "rb") as zf:
    zip_sha256 = hashlib.sha256(zf.read()).hexdigest()
with open(sha_path, "w") as sf:
    sf.write(f"SHA256: {zip_sha256}\n")

# Manual verification:
```

```
sha256sum wowscanner_archive_20260419_154232.zip
# Output must match wowscanner_archive_20260419_154232.sha256 exactly
```

Attack Resistance Matrix

Attack Scenario	Layer 1 .sha256	Layer 2 .crc	Layer 3 HMAC	Layer 4 archive
Edit 1 byte in .txt report	DETECT	—	DETECT	DETECT
Edit content.xml inside .odt, recompress	DETECT	DETECT	DETECT	DETECT
Delete a report file	DETECT	—	DETECT	DETECT
Replace report + forge its .sha256	bypass	—	DETECT	DETECT
Replace all reports + all .sha256 files	bypass	bypass	DETECT	DETECT
Rebuild ZIP + forge INTEGRITY.txt HMAC	bypass	bypass	DETECT (machine-id)	DETECT
Same machine: full rebuild + forge HMAC	hard	hard	machine-id must match	size/hash delta

Auth: The HMAC key = sha256(hostname + "|" + /etc/machine-id). This key is never stored and never sent over the network. It is derived identically at scan time and at verify time. An attacker on a different machine with full knowledge of the file contents still cannot compute a valid HMAC — the machine identity is the secret.

```
# cmd_verify() output when tampering is detected:
  [✘]  wowscanner_report_20260419_154232.odt  HASH MISMATCH
       Expected SHA-256: 7c3e9a1f8d4b2d7c...
       Actual   SHA-256: a1b2c3d4e5f6a7b8...
       File was modified after scan time.  ← tampering detected
```

Tip: Copy the archive ZIP and its .sha256 sidecar to an offline location immediately after each scan. Even if the server is later fully compromised and all local files overwritten, you retain a cryptographically verifiable record of what was found at scan time.

19. Scheduling & Automation

Systemd Weekly Timer — cmd_install_timer() L11391

```
sudo bash wowscanner.sh install-timer

# Creates:
# /etc/systemd/system/wowscanner.service  — ExecStart=bash /path/to/wowscanner.sh
# /etc/systemd/system/wowscanner.timer    — OnCalendar=weekly (Sun 00:00 UTC)

systemctl daemon-reload
systemctl enable --now wowscanner.timer
systemctl list-timers wowscanner.timer     # show next run time
journalctl -u wowscanner.service -n 200    # last run output

sudo bash wowscanner.sh remove-timer       # cmd_remove_timer() L11445
```

Cron Alternative

```
# /etc/cron.d/wowscanner
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
# Weekly Sunday 02:00 — fast mode, no pentest, quiet, email report:
0 2 * * 0  root  cd /opt/wowscanner && \
  bash wowscanner.sh --no-pentest --no-lynis --fast-only --quiet \
  --email=security@example.com >> /var/log/wowscanner-cron.log 2>&1
```

Email & Webhook Flags

```
sudo bash wowscanner.sh --email=security@example.com
sudo bash wowscanner.sh --webhook=https://hooks.slack.com/services/YOUR/HOOK

# Webhook JSON payload sent by deliver_report():
{"text":"Wowscanner scan on *webserver01.example.com*\nScore: *53%* (71/132) — MODERATE"}
```

20. Troubleshooting — 16 Scenarios with Code References

Symptom	Root Cause (Code Reference)	Exact Fix
Passphrase prompt when running reset-auth forgot	_ws_early_gate() pre-v2.3.0 (L700) checked CMD_RESET_AUTH variable only. Variable propagation failure → gate fires.	Replace with v2.3.0. New gate checks raw $@ in for loop before any variable lookup.
Wrong passphrase. N attempt(s) left.	_ws_prompt() (L644) comparison failed. Keyboard layout mismatch or forgotten.	Ctrl-C, then: sudo bash wowscanner.sh reset-auth forgot
Too many failed attempts. Access denied.	_ws_prompt() exits after 3 failures. Logs to auth_failures.log.	sudo bash wowscanner.sh reset-auth forgot (root = ownership proof, no old pass needed)
SECURITY ALERT: auth.key tampered	Dead _auth_gate() at L1949 in pre-v2.3.0 used undefined _AUTH_KEY var → SECURITY ALERT + exit. Removed in v2.3.0.	Replace with v2.3.0. _auth_prompt/_auth_gate/_auth_hash/_auth_crc all removed.
No output files after scan	set -eo pipefail triggered inside a Python heredoc in a generator function.	bash -x wowscanner.sh 2>&1 | tee /tmp/debug.log — find which heredoc failed.
Progress bar not shown	_progress_init() L15835: [[-c /dev/tty]] || return — /dev/tty unavailable in cron/pipe.	Expected. Bar silently disabled when not in terminal. REPORT still written normally.
ODT/ODS file will not open	Pre-v2.2.0: mimetype stored compressed in ZIP. ODF 1.3 requires it uncompressed (ZIP_STORED).	v2.3.0 fixed. Or run: bash wowscanner.sh verify to diagnose.
debsums shows no output / permission denied	debsums needs root to read /var/lib/dpkg/info/*.md5sums.	Run full scan as root: sudo bash wowscanner.sh
rkhunter: WARNING: file properties have changed	rkhunter database stale — binaries updated since last --propupd.	sudo rkhunter --propupd then re-scan.
Lynis takes > 5 minutes	LYNIS_FULL=true is set, runs without --quick --fast.	Unset LYNIS_FULL or use --no-lynis. Default is fast mode.
verify fails: HMAC mismatch	cmd_verify() machine_key = sha256(hostname+machine-id). Different machine → different key.	Run verify on the same machine that produced the archive.
LAN scan: no hosts found	arp-scan not installed; wrong interface detected; no network (VM/container).	sudo apt-get install arp-scan; check ip route for correct interface.
declare -A: command not found	bash version below 4.2 — no associative arrays.	sudo apt-get install --only-upgrade bash

Symptom	Root Cause (Code Reference)	Exact Fix
score_history.db grows indefinitely	archive_outputs() appends one line per scan — by design.	tail -365 /var/lib/wowscanner/score_history.db > /tmp/s && mv /tmp/s /var/lib/wowscanner/score_history.db
clean --all did not remove passphrase (pre-v2.2.0)	Pre-v2.2.0 cmd_clean() did not include rm -rf "$_WS_AUTH_DIR".	v2.3.0 fixed. Or manually: sudo rm -rf /etc/wowscanner/
wowscanner_recover.sh cannot delete auth.key	Script checks [[$(id -u) -ne 0]] → exit 1. Not running as root.	sudo bash wowscanner_recover.sh

21. Quick Reference Card

Essential Commands

```
sudo bash wowscanner.sh                          # Full 54-section audit
sudo bash wowscanner.sh --no-pentest            # Skip active probes (5-12 min saved)
sudo bash wowscanner.sh --no-pentest --no-lynis # Fast full audit (~2-4 min)
sudo bash wowscanner.sh --fast-only --quiet      # Fastest possible (~2 min)
sudo bash wowscanner.sh harden                  # Write + apply 21 sysctl params
sudo bash wowscanner.sh baseline                # Snapshot PASS findings
sudo bash wowscanner.sh diff                    # Compare last 2 scans
sudo bash wowscanner.sh verify                  # 4-layer integrity check
sudo bash wowscanner.sh clean --all             # Wipe auth + all history
sudo bash wowscanner.sh clean --full            # Complete factory reset
sudo bash wowscanner.sh reset-auth              # Interactive password menu
sudo bash wowscanner.sh reset-auth forgot       # Reset forgotten passphrase (root proof)
sudo bash wowscanner.sh reset-auth rk           # Reset via 48-char recovery key
bash  wowscanner.sh reset-auth keyid            # Show key fingerprint (no auth needed)
bash  wowscanner.sh reset-auth recover          # Unlock old archives via auth.key.prev
bash  wowscanner.sh example-output              # Sample output files (no scan, no auth)
```

Finding Level Reference

Symbol	Level	SCORE	TOTAL	Meaning
[✔ PASS]	PASS	+1	+1	Security control verified correctly configured
[✘ FAIL]	FAIL	+0	+1	Control absent or misconfigured — action required immediately
[■ WARN]	WARN	+0	+1	Suboptimal — review and address within 30 days
[■ INFO]	INFO	+0	+0	Informational context — no score effect, no action required
[■ SKIP]	SKIP	+0	+0	Tool missing or service not active — no score effect
[● NOTE]	NOTE	+0	+0	Contextual note for the following finding
■ ■	detail	+0	+0	Sub-line providing extra detail for the preceding finding

Password Recovery Decision Tree

Situation	Command to Run
Know current passphrase, want to change it	sudo bash wowscanner.sh reset-auth change
Forgot passphrase, have root SSH / console access	sudo bash wowscanner.sh reset-auth forgot
Forgot passphrase, have the 48-char recovery key	sudo bash wowscanner.sh reset-auth rk

Situation	Command to Run
Forgot passphrase, no root access, no recovery key	sudo bash wowscanner_recover.sh (deletes auth.key → triggers first-run wizard)
Wipe everything, reinstall from scratch	sudo bash wowscanner.sh reset-auth force [type: WIPE to confirm]
Check which key is currently active (no auth needed)	bash wowscanner.sh reset-auth keyid
Old archives unreadable after forgot/force/rk reset	bash wowscanner.sh reset-auth recover [enter OLD passphrase when prompted]

Top 10 Hardening Actions — Priority Order

- **1. sudo bash wowscanner.sh harden** — 21 sysctl parameters written and applied immediately.
- **2.** SSH: **PermitRootLogin prohibit-password** + **PasswordAuthentication no** + move to non-standard port (e.g. 2222).
- **3.** PAM: install **libpam-pwquality** + configure pam_faillock (5 failures = 15 min lockout).
- **4.** Databases: **bind-address=127.0.0.1** in mysqld.cnf; **bind 127.0.0.1 + requirepass** in redis.conf.
- **5.** Firewall: **ufw default deny incoming; ufw allow 2222/tcp; ufw allow 443/tcp; ufw enable**.
- **6.** Updates: **apt-get upgrade** immediately; enable **unattended-upgrades** security-only mode.
- **7.** Compilers: **apt-get purge gcc g++ clang build-essential** — never needed on production.
- **8.** Secrets: move .env and config files to **/etc/** (outside web root), **chmod 600**.
- **9.** Boot: **grub-mkpasswd-pbkdf2** then add to /etc/grub.d/40_custom and run update-grub.
- **10.** AppArmor: **sudo aa-enforce /etc/apparmor.d/*** — move all profiles to enforce mode.

Wowscanner v2.3.0 · 16,495 lines · 137 functions · 54 sections · 925-point progress · 4-layer integrity · AES-256-CTR + PBKDF2-HMAC-SHA256 · cocoflan 2026

Appendix A — section_summary() Complete Terminal Output (L7538)

section_summary() is dispatched as `_progress_start "16 portscan"; section_portscan || true; _elapsed "16 portscan"` then immediately followed by section_summary() outside the normal dispatch loop — it is called directly from main() after all 54 sections complete. It reads the $REPORT file directly via grep to compute counts, then renders the final assessment panel. Below is the complete verbatim terminal output from a MODERATE-rated scan.

```
═══════════════════════════════════════════════════════════════════════════
■                                                                         ■
■  17. SUMMARY                                                            ■
■                                                                         ■
═══════════════════════════════════════════════════════════════════════════

  ═══════════════════════════════════════════════════════════════════════
  ■  ■  SECURITY ASSESSMENT COMPLETE                                  ■
  ═══════════════════════════════════════════════════════════════════════
  ■  ═══════════════════════════════════════  53%                     ■
  ■  ■  Passed 71 of 132 checks  ·  MODERATE                         ■
  ■  ✘ 12 FAIL  ■ 24 WARN  ✔ 71 PASS  ■ 47 INFO                      ■
  ■  Several issues need attention. Address FAIL items as a priority. ■
  ═══════════════════════════════════════════════════════════════════════

  Output files (current directory):
  ═══════════════════════════════════════════════════════════════════
  ■  wowscanner_20260419_154232.txt
  ■    Plain-text audit log (full scan output)
  ■  wowscanner_report_20260419_154232.odt
  ■    Graphical report — open with LibreOffice Writer
  ■  wowscanner_report_20260419_154232.html
  ■    Self-contained HTML report — open in any browser
  ■  wowscanner_stats_20260419_154232.ods
  ■    Statistics workbook — open with LibreOffice Calc
  ■  wowscanner_intel_20260419_154232.odt
  ■    Intelligence report with CVE context
  ■  wowscanner_archive_20260419_154232.zip
  ■    Signed archive (HMAC + dual SHA-256/SHA-512 manifest)
  ═══════════════════════════════════════════════════════════════════

  ■■ RECOMMENDED NEXT STEPS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  ■
  ■  ①  Fix all FAIL items immediately
  ■  ②  Review WARN items — apply where relevant
  ■  ③  sudo bash wowscanner.sh baseline        — snapshot PASS findings
  ■  ④  sudo bash wowscanner.sh harden          — apply sysctl hardening
  ■  ⑤  sudo bash wowscanner.sh install-timer   — weekly automated scans
  ■  ⑥  less wowscanner_findings_20260419_154232.txt  — browse findings
  ■  ⑦  Aim for Lynis hardening index >= 80
  ■  ⑧  Set up monitoring: auditd + fail2ban + OSSEC
  ■
  ═══════════════════════════════════════════════════════════════════════
```

Info: The score bar uses Unicode block characters: filled = ■ (U+2588 FULL BLOCK), empty = ■ (U+2591 LIGHT SHADE). Falls back to ASCII # and - when the terminal is not UTF-8 (detected from LC_ALL/LC_CTYPE/LANG). The bar width adapts: `_bar_w = (_PROGRESS_COLS - 4) - 20`, clamped to [10, _w-20]. At 80 columns: bar_w = 56 characters.

Score Counting — Exact Code Path Through pass() / fail() / warn()

```
# Every check in every section_*() function calls exactly one of:
pass()   { SCORE=$((SCORE+1)); TOTAL=$((TOTAL+1)); log "  ${BGREEN}[✔ PASS]${NC}  $1  ${BGREEN}Ω${NC}"; }
fail()   {                     TOTAL=$((TOTAL+1)); log "  ${BRED}[✘ FAIL]${NC}  $1  ${BRED}Ω${NC}"; }
warn()   {                     TOTAL=$((TOTAL+1)); log "  ${YELLOW}[■ WARN]${NC}  $1  ${YELLOW}Ω${NC}"; }
info()   { log "  ${BCYAN}[■ INFO]${NC}  $1  ${BCYAN}Ω${NC}"; }   # no SCORE or TOTAL effect
skip()   { log "  ${DIM}[■ SKIP]${NC}  $1  ${DIM}Ω${NC}"; }       # no SCORE or TOTAL effect
detail() { log "  ${DIM}■${NC}       ${BMAGENTA}■${NC} $1"; }      # sub-detail, no effect

# section_summary() counts them from the finished REPORT file:
_n_fail=$(grep -c "\[✘ FAIL\]" "$REPORT" 2>/dev/null || echo 0)
_n_warn=$(grep -c "\[■ WARN\]" "$REPORT" 2>/dev/null || echo 0)
_n_pass=$(grep -c "\[✔ PASS\]" "$REPORT" 2>/dev/null || echo 0)
_n_info=$(grep -c "\[■ INFO\]" "$REPORT" 2>/dev/null || echo 0)
# These are display-only — SCORE and TOTAL are the authoritative variables
```

Appendix B — _ws_write_auth_key() Complete Flow (L407)

_ws_write_auth_key() is called in exactly three situations: (1) First-run wizard: _ws_first_run_setup() calls it with empty second arg → fresh data_key. (2) reset-auth change: called with old data_key as second arg → data_key preserved, passphrase changes. (3) reset-auth forgot/force/rk: called with empty second arg → new data_key generated (old archives inaccessible). The function is the only place in the codebase that writes auth.key. Every field is freshly generated on every call — salt, iv, dk_iv are always new random bytes.

```
_ws_write_auth_key() {   # L407
  local _pass="$1"
  local _existing_dk="${2:-}"    # empty = generate new data_key; non-empty = carry across
  local _rounds=200000

  # Step 1: Generate fresh random material
  _salt=$(_ws_crypto randombytes 32)   # 64 hex chars = 32 bytes of os.urandom
  _iv=$(_ws_crypto randombytes 16)     # 32 hex chars = 16 bytes of os.urandom

  # Step 2: Derive master key from passphrase via PBKDF2
  _key=$(_ws_crypto hash "$_pass" "$_salt" "$_rounds")
  # → pbkdf2_hmac("sha256", pw.encode(), unhexlify(salt), 200000, dklen=32).hex()

  # Step 3: Derive recovery key (48 hex chars) from master key
  _rk_val=$(_ws_crypto recovery-key "$_key")
  # → hashlib.sha256(master_key_bytes + b"RECOVERY").hexdigest()[:48]
  _rk_hash=$(python3 -c "import hashlib; print(hashlib.sha256(
      "${_rk_val}".encode()).hexdigest())")
  export WS_RECOVERY_KEY="$_rk_val"   # shown once in first-run wizard

  # Step 4: AES-CTR encrypt sentinel "WOWSCANNER_OK\n\x00\x00"
  _ct=$(_ws_crypto encrypt "$_pass" "$_salt" "$_rounds" "$_iv")

  # Step 5: Generate or carry data_key
  if [[ -n "$_existing_dk" ]]; then
    _data_key="$_existing_dk"         # change passphrase: preserve key → archives readable
  else
    _data_key=$(_ws_crypto randombytes 32)   # fresh key: old archives inaccessible
  fi
  _key_id=$(_ws_crypto keyid "$_data_key")   # sha256(data_key)[:16] = fingerprint

  # Step 6: Wrap data_key under master_key
  _dk_iv=$(_ws_crypto randombytes 16)         # fresh IV for wrapping
  _wrapped_dk=$(_ws_crypto wrap "$_key" "$_dk_iv" "$_data_key")
  # → aes_cbc(master_key, dk_iv, data_key) — encrypted data_key

  # Step 7: HMAC covers all crypto fields
  _hmac_data="${_salt}${_iv}${_ct}${_dk_iv}${_wrapped_dk}"  # hex concat
  _hmac_val=$(_ws_crypto hmac "$_key" "$_hmac_data")

  # Step 8: Write auth.key atomically (printf is a single write)
  printf "%s\n" \
    "# Wowscanner auth v2" \
    "# DO NOT EDIT — change via: bash ${_script_path} reset-auth" \
    "ver=2" \
    "rounds=${_rounds}" \
    "salt=${_salt}" \
    "iv=${_iv}" \
    "hmac=${_hmac_val}" \
```

```
    "cipher=${_ct}" \
    "dk_iv=${_dk_iv}" \
    "dk_wrapped=${_wrapped_dk}" \
    "key_id=${_key_id}" \
    "rk_hash=${_rk_hash}" \
    > "$_WS_AUTH_KEY"
  chmod 400 "$_WS_AUTH_KEY"         # read-only, owner=root only
  chown root:root "$_WS_AUTH_KEY"   # sudo required to read
}
```

Auth: The HMAC field covers: salt + iv + cipher + dk_iv + dk_wrapped (hex-concatenated as bytes). This means tampering with ANY of those fields is detected on the next verify attempt. The rk_hash is NOT covered by the HMAC — it is derived from the master key and is independently verifiable. A corrupted rk_hash disables recovery key verification but does not affect passphrase authentication.

Appendix C — deliver_report() & archive_outputs() Full Code (L11531, L15460)

deliver_report() L11531 — Webhook Payload + Email MIME Construction

```
deliver_report() {   # called from main() after all generators finish
  local _score="$1" _total="$2" _pct="$3" _archive="$4"

  # ■■ Webhook (Slack / Teams compatible JSON) ■■■■■■■■■■■■■■■■■■■■■■■■■■■
  if [[ -n "$WEBHOOK_URL" ]]; then
    local _rating="GOOD"
    [[ "$_pct" -lt 80 ]] && _rating="MODERATE"
    [[ "$_pct" -lt 50 ]] && _rating="CRITICAL"
    _payload=$(printf '{"text":"Wowscanner scan complete on *%s*\nScore: *%d%%* (%d/%d) — %s\nArchive: %s"}' \
      "$(hostname -f 2>/dev/null || hostname)" "$_pct" "$_score" "$_total" "$_rating" \
      "$(basename "$_archive")")
    if curl -s -o /dev/null -w "%{http_code}" \
         -H "Content-Type: application/json" \
         -d "$_payload" --max-time 10 "$WEBHOOK_URL" 2>/dev/null | grep -q "^2"; then
      info "Webhook notification sent to: ${WEBHOOK_URL%%//*}/..."
    else
      warn "Webhook delivery failed — check WEBHOOK_URL"
    fi
  fi

  # ■■ Email (msmtp / sendmail / ssmtp) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  if [[ -n "$REPORT_EMAIL" ]]; then
    local _mailer=""
    for _m in msmtp sendmail ssmtp; do
      command -v "$_m" &>/dev/null && { _mailer="$_m"; break; }
    done
    if [[ -z "$_mailer" ]]; then
      warn "REPORT_EMAIL set but no mailer found (msmtp/sendmail/ssmtp)"
    elif [[ -f "$_archive" ]]; then
      local _arc_sz; _arc_sz=$(stat -c%s "$_archive" 2>/dev/null || echo 0)
      if [[ "$_arc_sz" -gt 20971520 ]]; then   # > 20 MB: no attachment
        warn "Archive is $(( _arc_sz / 1048576 ))MB — too large to email"
        # ... sends summary-only email
        return 0
      fi
      # Builds multipart MIME with base64-encoded ZIP attachment
      { echo "To: $REPORT_EMAIL"
        echo "Subject: Wowscanner — $(hostname -f) — ${_pct}%"
        echo "MIME-Version: 1.0"
        echo "Content-Type: multipart/mixed; boundary=\"wowboundary\""
        echo ""; echo "--wowboundary"
        echo "Content-Type: text/plain; charset=utf-8"; echo ""
        echo "Wowscanner scan on $(hostname -f)"
        echo "Score: ${_pct}% (${_score}/${_total})  Rating: $_rating"
        echo "--wowboundary"
        echo "Content-Type: application/zip"
        echo "Content-Disposition: attachment; filename=\"$(basename "$_archive")\""
        echo "Content-Transfer-Encoding: base64"; echo ""
        base64 "$_archive"
        echo "--wowboundary--"
      } | "$_mailer" "$REPORT_EMAIL" 2>/dev/null \
```

```
        && info "Report emailed to: ${REPORT_EMAIL}" \
        || warn "Email delivery failed — check mailer configuration"
    fi
  fi
}
```

archive_outputs() L15460 — 14-Step Python Heredoc

```
archive_outputs() {   # called from main() after deliver_report()
  local _dir="$PWD"
  local _zip="${_dir}/wowscanner_archive_${TIMESTAMP}.zip"
  local _sha="${_dir}/wowscanner_archive_${TIMESTAMP}.sha256"

  # 1. Write .txt/.findings CRC sidecars FIRST (bundled inside ZIP)
  [[ -f "${_dir}/wowscanner_${TIMESTAMP}.txt" ]]          && write_txt_crc "..."
  [[ -f "${_dir}/wowscanner_findings_${TIMESTAMP}.txt" ]] && write_txt_crc "..."

  # 2. Collect files — only those matching TIMESTAMP
  while IFS= read -r -d "" f; do _files+=("$f"); done < <(
    find "$_dir" -maxdepth 1 -type f \(
      -name "wowscanner_${TIMESTAMP}.txt"         \
      -o -name "wowscanner_*.txt.sha256"          \
      -o -name "wowscanner_findings_*.txt"         \
      -o -name "wowscanner_report_${TIMESTAMP}.odt" \
      -o -name "wowscanner_report_${TIMESTAMP}.html" \
      -o -name "wowscanner_stats_${TIMESTAMP}.ods" \
      -o -name "wowscanner_intel_${TIMESTAMP}.odt" \
    \) -print0)

  # 3-12. Python heredoc: dual-hash each file, build INTEGRITY.txt,
  #       derive machine_key, compute HMAC, seal ZIP, write sidecar
  python3 - "$_zip" "$_sha" "$PERSIST_DIR" "${_files[@]}" << 'ARCHEOF'
  # machine_key = sha256(hostname + "|" + /etc/machine-id)
  # INTEGRITY.txt: one line per content file with sha256+sha512+size+mtime+mode+uid+gid
  # Last line: "# HMAC-SHA256: <64-hex>" covers all preceding lines
  # ZIP built with ZIP_DEFLATED; INTEGRITY.txt stored as ZIP_STORED (uncompressed)
  # .sha256 and .crc sidecars bundled in ZIP but not listed in INTEGRITY.txt
ARCHEOF

  # 13. Compute sha256 of the sealed ZIP
  python3 -c "import hashlib,sys; h=hashlib.sha256(); ..."

  # 14. Call record_score_history(SCORE, TOTAL, PERCENTAGE)
  record_score_history "$SCORE" "$TOTAL" "$PERCENTAGE"
```

Appendix D — All 62 Section Functions — Complete Index

62 section functions are defined in the script (vs 54 dispatched from main()). The 8 extra functions are called internally by other sections or by specific commands. All 62 are listed below with their exact line number and calling context.

Function	L#	Called From	Notes
section_pentest_enum	L2139	main() dispatch 0a	nmap -A + enum4linux
section_pentest_web	L2361	main() dispatch 0b	nikto per HTTP/HTTPS port
section_pentest_ssh	L2413	main() dispatch 0c	hydra brute force
section_pentest_sqli	L2481	main() dispatch 0d	sqlmap basic probe
section_pentest_stress	L2527	main() dispatch 0e	stress-ng + hping3
section_sysinfo	L2688	main() dispatch 1	OS info, disk, memory, zombie
section_updates	L2798	main() dispatch 2	apt upgradable, security, unattended-upgrades
section_users	L2860	main() dispatch 3	UID0, empty pw, inactive, auth_keys
section_password_policy	L3014	main() dispatch 4	login.defs, PAM, shadow
section_ssh	L3078	main() dispatch 5	sshd_config, sshd -T
section_firewall	L3250	main() dispatch 6	ufw/iptables/nft
section_ports	L3298	main() dispatch 7	ss/netstat, RISKY_PORTS, db binding
section_permissions	L3417	main() dispatch 8	stat expected perms, WW files, SUID
section_services	L3540	main() dispatch 9	failed units, legacy services, inetd
section_logging	L3607	main() dispatch 10	syslog, auditd, fail2ban
section_kernel	L3772	main() dispatch 11	sysctl 20+ params, modules
section_cron	L3843	main() dispatch 12	crontab perms, WW scripts, PATH dirs
section_packages	L3989	main() dispatch 13	debsums, dpkg --audit, AIDE
section_hardening_advanced	L4213	main() dispatch 13c	688 lines: sudoers, certs, TLS, caps, DNS
section_network_container	L4906	main() dispatch 13d	private keys, .env, docker socket, LD_PRELOAD
section_chkrootkit	L5137	main() dispatch 14b	chkrootkit + rkhunter
section_mac	L5370	main() dispatch 14	AppArmor/SELinux
section_lynis	L5423	main() dispatch 15	lynis audit system --quick --fast

Function	L#	Called From	Notes
section_lan_scan	L6986	main() dispatch 16a	arp-scan/nmap/arp, JSON output
section_portscan	L7358	main() dispatch 16	nmap full port, KNOWN_RISKY, port_history.db
section_summary	L7538	main() after all 54	Score panel, output list, next steps
section_disk_encryption	L10918	section_hardening_advanced	LUKS/dm-crypt/swap encryption
section_capabilities	L10978	section_hardening_advanced	getcap -r / dangerous caps
section_dns_security	L11029	section_hardening_advanced	DoT, DNSSec, stub resolver
section_alt_containers	L11103	section_hardening_advanced	podman rootless, snap confinement
section_supply_chain	L11761	section_hardening_advanced	pip/npm audit, pip --trusted-host
section_immutable	L11855	section_hardening_advanced	chattr +i on critical files
section_proc_exposure	L11922	section_hardening_advanced	/proc leaks, hidepid, dmesg restrict
section_failed_logins	L13338	main() dispatch 17b	auth.log, fail2ban, top attackers
section_env_security	L13425	main() dispatch 17c	umask, PATH, LD_PRELOAD, startup files
section_usb_devices	L13496	main() dispatch 17d	lsusb, usb_storage, usbguard
section_world_writable_deep	L13548	main() dispatch 17e	find WW, sticky bit, SUID in /home
section_cert_audit	L13596	main() dispatch 17f	openssl enddate, SSH host key bits
section_network_security	L13665	main() dispatch 17g	ARP filter, redirects, TCP timestamps
section_auditd_check	L13747	main() dispatch 17h	auditd rules, logrotate coverage
section_open_files_check	L13810	main() dispatch 17i	lsof deleted, fd exhaustion, WW sockets
section_memory_security	L13904	main() dispatch 17j	swap encrypt, overcommit, kptr_restrict
section_pam_security	L13955	main() dispatch 17k	debsums pam, 2FA, pam_tty_audit
section_filesystem_hardening	L14002	main() dispatch 17l	/tmp noexec, /dev/shm, hidepid, home perms

Function	L#	Called From	Notes
section_container_security	L14071	main() dispatch 17m	docker daemon.json, socket perms, privileged
section_repo_security	L14143	main() dispatch 17n	trusted=yes, apt-key, third-party repos
section_time_sync	L14199	main() dispatch 17o	NTP sync, chronyc offset
section_ipv6_security	L14257	main() dispatch 17p	ip6tables, accept_ra, forwarding
section_ssh_extras	L14316	main() dispatch 17q	ciphers, MACs, KEX algorithms
section_core_dump_security	L14411	main() dispatch 17r	ulimit, suid_dumpable, core_pattern
section_systemd_hardening	L14470	main() dispatch 17s	per-service sandboxing scoring
section_sudo_audit	L14551	main() dispatch 17t	NOPASSWD, visudo, timestamp_timeout
section_log_integrity	L14621	main() dispatch 17u	remote logging, logrotate auth coverage
section_compiler_tools	L14699	main() dispatch 17v	gcc/g++/clang/gdb on prod server
section_network_interfaces	L14799	main() dispatch 17w	promiscuous, ARP poisoning, error counters
section_kernel_modules	L14871	main() dispatch 17x	lsmod vs _risky_desc, module_sig_enforce
section_mac_profiles	L14953	main() dispatch 17y	aa-status, complain-mode profiles
section_exposure_summary	L15023	main() dispatch 17z	deduplicated port risk summary
section_hardware_security	L15122	main() dispatch 17b3	Secure Boot, IOMMU, CPU vulns, TPM
section_boot_security	L15202	main() dispatch 17b4	GRUB password, cmdline audit
section_web_server	L15272	main() dispatch 17b5	nginx/apache server_tokens, TLS, headers
section_secrets_scan	L15373	main() dispatch 17b6	private keys, .env, bash_history, agent sockets

Appendix E — wowscanner_stats_[TS].ods — All 8 Sheets in Detail

Generated by generate_stats_ods() (L8500). The Python heredoc STATSEOF writes a valid ODS 1.3 container using zipfile.ZipFile. All cell values are written as with office:value-type and office:value. Each sheet is a element within the single content.xml. Charts in the Charts sheet use SVG embeds, identical to the ODT charts.

Sheet	Columns	Row Count	Content
1. Overview	Metric, Value	18 rows	Host, OS, kernel, scan date, score, total, percentage, rating, sections run, FAIL count, WARN count, PASS count, INFO count, SKIP count, scan duration, lynis index, rkhunter result, chkrootkit result.
2. Per-Section	Section, Label, Function, Weight, FAILs, WARNs, PASSes, INFOs, SKIPs	54 rows	One row per section. FAILs/WARNs/PASSes parsed from $REPORT by section header. Weight from _PROGRESS_WEIGHTS[]. Colour coding: red rows have FAIL > 0.
3. All Findings	Type, Message, Section, Line	Varies	Every PASS, FAIL, WARN, INFO from $REPORT. Type column = PASS/FAIL/WARN/INFO. Sortable by Type to group all FAILs together.
4. All Issues	Port, Protocol, Service, State, First Seen, Count	Varies	Content of port_history.db reformatted as table. Pivots all historical port issue data across all runs.
5. FAIL Deep-Dive	Message, Section, Priority, Remediation	Varies	Every FAIL from current scan with auto-generated remediation text. Priority (Critical/High/Medium/Low) based on message keyword matching. Sorted by Priority.
6. WARN Deep-Dive	Message, Section, Category	Varies	Every WARN from current scan. Category column classifies: Network/Auth/Crypto/Logging/File/Service.
7. ChartData	Various	Fixed	Raw data for chart rendering: score trend (epoch + pct), section severity (label + fail + warn + pass), finding distribution (type + count), systemd sandboxing scores.
8. Charts	Embedded SVG	4 charts	Score Trend Line, Section Severity Heatmap, Finding Distribution Donut, Systemd Sandboxing Bar. All SVGs generated in Python, identical to ODT versions.

wowscanner_intel_[TS].odt — Intelligence Report Structure

Generated by generate_odf_intel_report() (L5682). Focuses on threat context and remediation rather than raw findings. Produced from the same scan data but cross-referenced with embedded CVE and threat intel datasets.

Section	Content
Executive Summary	One-paragraph risk summary with GOOD/MODERATE/CRITICAL rating, host info, and top 3 priorities.

Section	Content
CVE Context	For each FAIL finding: maps to known CVE classes (SSH weak ciphers → CVE-2016-1908 class, etc.). CVE mappings are static, embedded in the script — not fetched at runtime.
CISA KEV Cross-Reference	Compares detected FAIL conditions against CISA Known Exploited Vulnerabilities catalogue patterns. Known-exploited conditions are flagged in red with KEV reference.
Attack Surface Matrix	Rendered as a 6×6 grid: Attack vector vs Exploitability vs Impact. Each current FAIL is plotted on the matrix. Higher-right quadrant = immediate action.
Remediation Priority Matrix	Table of all FAILs sorted by: Impact * Exploitability * Ease-of-fix. Top 8 items highlighted. Specific command-line fix shown for each.
Appendix: Raw Section Data	Abbreviated version of the full audit log for reference context.

Appendix F — Annotated Full Scan Transcript — Real Terminal Output

The following is a complete, annotated scan transcript from Ubuntu 22.04 LTS (webserver01.example.com) with a real MODERATE result. Every line is traced to the specific function that generated it. Progress bar line shown before each section header.

Pre-Scan: Auth Gate + Startup Banner

```
$ sudo bash wowscanner.sh --no-pentest

# _ws_prompt() (L644) prints recovery hint before asking:
  Wowscanner v2.3.0  — passphrase required

  Forgot your passphrase?  Run this instead:
  sudo bash wowscanner.sh reset-auth forgot

  Passphrase: ■■■■■■■■■■■■■   [entered, not echoed]

# _ws_verify_pass() (L502): PBKDF2 derives key, HMAC verified, sentinel OK
# WS_DATA_KEY and WS_KEY_ID exported

# main() (L15975): Startup banner via log() + tee:
■■■■■■■■■■■■■
```

```
[■ INFO]  OS: Ubuntu 22.04.4 LTS (jammy)  Ω
[■ INFO]  Kernel: 5.15.0-112-generic  Ω
[■ INFO]  Architecture: x86_64  Ω
[■ INFO]  Uptime: 14 days, 3:22  Ω

■■ Disk & memory ■■■
```

```
[✔ PASS]  No world-writable files found  Ω
[✔ PASS]  No world-writable directories without sticky bit  Ω

■■ SUID/SGID binaries ■■■
```

Post-Scan: Output Generators + Archive (after all 54 sections)

```
# main() calls each generator in order:
  [■]  Generating ODT report → wowscanner_report_20260419_154232.odt ...
  [■]  Generating HTML report → wowscanner_report_20260419_154232.html ...
  [■]  Generating ODS stats  → wowscanner_stats_20260419_154232.ods ...
  [■]  Generating Intel ODT  → wowscanner_intel_20260419_154232.odt ...
  [■]  Generating findings   → wowscanner_findings_20260419_154232.txt ...

  # archive_outputs() (L15460):
  ■■ Archiving scan outputs ■■■
```

Appendix G — Flag Interaction Reference & Full Environment Variable List

All flags are parsed in the main arg loop at L244. Boolean flags set a USE_* or CMD_* variable then continue to the next argument. Value flags (--email=, --webhook=) extract the value via `"${flag#*=}"`. The dispatcher at L16380+ checks CMD_* flags in priority order before falling through to main(). Scan flags (USE_*) are checked in main() via _recompute_progress_cum() before _progress_init() runs.

Flag	Internal Variable	Type	Default	Code Path
--no-pentest	USE_PENTEST	bool	true	L203. _recompute_progress_cum zeros indices 0-4. Sections 0a-0e skipped via: [["$USE_PENTEST" != "true"]] && skip "..." && return
--no-lynis	USE_LYNIS	bool	true	L202. Index 22 zeroed. section_lynis(): [["$USE_LYNIS" != "true"]] && skip ... && return
--no-rkhunter	USE_RKHUNTER	bool	true	L204. Index 20 zeroed. section_chkrootkit(): skips rkhunter sub-run only (chkrootkit still runs).
--no-hardening	USE_HARDENING	bool	true	L205. Index 18 zeroed. section_hardening_advanced() immediately returns.
--no-netcontainer	USE_NETCONTAINER	bool	true	L206. Index 19 zeroed. section_network_container() immediately returns.
--fast-only	USE_FAST_ONLY	bool	false	L207. No weight change. Each section checks [["$USE_FAST_ONLY" == "true"]] and skips slow sub-operations (e.g. deep find, rkhunter full scan).
--quiet	QUIET	bool	false	L208. info() becomes no-op: [["$QUIET" == "true"]] && return. SCORE/TOTAL/REPORT unaffected.
--email=addr	REPORT_EMAIL	str	empty	L239. Passed to deliver_report(). Prefers --email flag over REPORT_EMAIL env var.
--webhook=url	WEBHOOK_URL	str	empty	L240. Passed to deliver_report(). Prefers --webhook flag over WEBHOOK_URL env var.
clean	CMD_CLEAN	bool	false	L266. Dispatcher calls cmd_clean(). --all/--full/--integrity parsed inside cmd_clean() from remaining $@.
verify	CMD_VERIFY	bool	false	L268. Dispatcher calls cmd_verify().
diff	CMD_DIFF	bool	false	L270. Dispatcher calls cmd_diff().

Flag	Internal Variable	Type	Default	Code Path
harden	CMD_HARDEN	bool	false	L272. Dispatcher calls cmd_harden().
baseline	CMD_BASELINE	bool	false	L274. Dispatcher calls cmd_baseline().
reset-auth	CMD_RESET_AUTH	bool	false	L276. Dispatcher calls cmd_reset_auth() with $@ forwarded.
example-output	CMD_EXAMPLE_OUTPUT	bool	false	L278. Dispatcher calls cmd_example_output(). No auth check — gate returns 0 for CMD_EXAMPLE_OUTPUT.
install-timer	CMD_INSTALL_TIMER	bool	false	L280. Dispatcher calls cmd_install_timer().
remove-timer	CMD_REMOVE_TIMER	bool	false	L282. Dispatcher calls cmd_remove_timer().
install-completion	CMD_INSTALL_COMP	bool	false	L284. Dispatcher calls cmd_install_completion().
--help / -h	CMD_HELP	bool	false	L287. _ws_early_gate() returns 0 for CMD_HELP. Dispatcher calls cmd_help() which prints full panel.

All Environment Variable Overrides

Environment Var	Default	How Overridden by Script	Effect
LYNIS_FULL	unset	export LYNIS_FULL=true sudo -E bash ...	Removes --quick --fast from lynis invocation. Adds 2-10 minutes.
RKH_FULL	unset	export RKH_FULL=true sudo -E bash ...	Removes --skip-keyfile-check from rkhunter. Adds 1-5 minutes.
APT_CACHE_MAX_AGE	86400	export APT_CACHE_MAX_AGE=0 sudo bash ...	0 = force apt-get update before every install. Useful in CI.
WEBHOOK_URL	empty	export WEBHOOK_URL=https://... (or --webhook= flag)	JSON POST to Slack/Teams/generic endpoint after scan.
REPORT_EMAIL	empty	export REPORT_EMAIL=addr (or --email= flag)	MIME multipart email with ZIP attachment. Needs msmtp/sendmail.
N_TIMING_BASELINE	8	export N_TIMING_BASELINE=20 sudo -E bash ...	How many past runs to include in EWMA ETA computation.
COLUMNS	auto	export COLUMNS=120 sudo -E bash ...	Override terminal width detection (fallback when stty fails).

Environment Var	Default	How Overridden by Script	Effect
LINES	auto	export LINES=50 sudo -E bash ...	Override terminal height detection.
WS_DATA_KEY	auto	exported by _ws_verify_pass()	Set internally. Reading externally: shows key_id fingerprint for archive matching.
WS_KEY_ID	auto	exported by _ws_verify_pass()	Set internally. Reading externally: sha256(data_key).hexdigest()[:16].
WS_RECOVERY_KEY	auto	exported by _ws_write_auth_key()	Set internally at first-run only. Shown in wizard. Never persisted.

Appendix H — Findings Report Format & Parser (L13099)

generate_findings_report() (L13099) produces wowscanner_findings_[TS].txt. It reads $REPORT via sed, extracts FAIL and WARN lines, groups them by section, and writes a paginated summary with a priority-ordered remediation section. The file is also parsed by show_finding_delta() for regression detection across runs.

```
═══════════════════════════════════════════════════════════════════════
■  WOWSCANNER FINDINGS REPORT                                        ■
■  webserver01.example.com  ·  2026-04-19 15:42:32 UTC               ■
■  Score: 53%  ·  MODERATE  ·  12 FAIL  24 WARN                      ■
═══════════════════════════════════════════════════════════════════════

═══════════════════════════════════════════════════════════════════════
 SECTION 7 — OPEN NETWORK PORTS
═══════════════════════════════════════════════════════════════════════
  [✘ FAIL]  Risky port open: 3306 (mysqld — bind to 127.0.0.1)
  [✘ FAIL]  Risky port open: 6379 (redis — bind to 127.0.0.1)
  [✘ FAIL]  Risky port open: 8080 (node — not in expected services)

═══════════════════════════════════════════════════════════════════════
 SECTION 11 — KERNEL PARAMETERS
═══════════════════════════════════════════════════════════════════════
  [■ WARN]  Potentially unused kernel module loaded: usb_storage
  [✘ FAIL]  kernel.kptr_restrict = 0 (expected 2)
  [✘ FAIL]  kernel.dmesg_restrict = 0 (expected 1)

═══════════════════════════════════════════════════════════════════════
 REMEDIATION PRIORITY
═══════════════════════════════════════════════════════════════════════
  #1 [CRITICAL]  Risky port open: 3306
       Fix: echo "bind-address = 127.0.0.1" >> /etc/mysql/mysql.conf.d/mysqld.cnf
            systemctl restart mysql
  #2 [CRITICAL]  Risky port open: 6379
       Fix: add "bind 127.0.0.1" to /etc/redis/redis.conf
            systemctl restart redis
  #3 [HIGH]  kernel.kptr_restrict = 0
       Fix: sudo bash wowscanner.sh harden
            (applies all 21 sysctl params including this one)
  ...
```

Ω-Delimiter — Machine Parsing Protocol

Every PASS/FAIL/WARN/INFO/SKIP line ends with a fixed Ω character (U+03A9, UTF-8: 0xCE 0xA9). This is the machine-parseable record delimiter used by all parsers in the codebase. Three independent parsers use it:

Parser Function	Pattern	Purpose
show_finding_delta() L11248	sed -n 's/.*\[✘ FAIL\] //;s/ Ω.*//p'	Extracts FAIL text for regression diff (comm -23 vs prev snapshot)
generate_findings_report() L13099	grep -oP '(?<=\[✔ PASS\]).+?(?= Ω)'	Extracts clean finding text for findings report sections
cmd_diff() L11322	grep -oP '(?<=\[✘ FAIL\]).+?(?= Ω)' file	Extracts FAILs from both reports for side-by-side comparison

Note: The ANSI stripping regex `sed "s/\[[0-9;]*m//g"` is always applied before set-difference operations. This ensures BRED [1;31m and YELLOW [1;33m colour codes do not make otherwise-identical findings appear different in the comm comparison.

Appendix I — Integration with SIEM, SOC & CI/CD

Splunk Integration

```
# 1. Add to Splunk Universal Forwarder inputs.conf:
[monitor:///var/lib/wowscanner/port_issues.log]
sourcetype = wowscanner_port
index = security

[monitor:///opt/wowscanner/wowscanner_*.txt]
sourcetype = wowscanner_audit
index = security

# 2. Splunk search to alert on NEW port issues:
index=security sourcetype=wowscanner_port state=NEW
| stats count by Port Protocol Service
| where count > 0
| eval alert_level=if(Service IN ("mysqld","redis","mongodb"),"CRITICAL","HIGH")

# 3. Webhook to Splunk HEC:
sudo bash wowscanner.sh --webhook=https://splunk-hec:8088/services/collector
```

Elastic / OpenSearch Integration

```
# 1. Filebeat configuration:
filebeat.inputs:
  - type: log
    paths: ["/var/lib/wowscanner/port_issues.log"]
    tags: ["wowscanner","security"]
    fields:
      log_type: wowscanner_port

# 2. Index pattern: wowscanner-*
# 3. Kibana alert: FAIL count > previous run FAIL count
```

GitLab CI / GitHub Actions Integration

```
# .github/workflows/wowscanner.yml:
name: Weekly Security Scan
on:
  schedule:
    - cron: "0 2 * * 0"   # Sunday 02:00 UTC
jobs:
  scan:
    runs-on: [self-hosted, linux, ubuntu-22.04]
    steps:
      - name: Run Wowscanner
        run: |
          cd /opt/wowscanner
          sudo bash wowscanner.sh --no-pentest --fast-only --quiet \
            --webhook=${{ secrets.SLACK_WEBHOOK }}
      - name: Upload artifacts
```

```
      uses: actions/upload-artifact@v4
      with:
        name: wowscanner-report
        path: wowscanner_*.{txt,zip,odt,html}
```

Score Threshold Gate — CI/CD Fail Condition

```
#!/bin/bash
# CI gate: fail build if CRITICAL rating (score < 50%)

sudo bash wowscanner.sh --no-pentest --quiet 2>&1

# Extract score from score_history.db:
SCORE_LINE=$(tail -1 /var/lib/wowscanner/score_history.db)
PCT=$(echo "$SCORE_LINE" | cut -d"|" -f4)

echo "Wowscanner score: ${PCT}%"
if [[ "$PCT" -lt 50 ]]; then
  echo "CRITICAL security rating — blocking deployment"
  exit 1
elif [[ "$PCT" -lt 80 ]]; then
  echo "MODERATE rating — deployment allowed, review findings"
  exit 0
else
  echo "GOOD rating — deployment approved"
  exit 0
fi
```